



ARCHITETTURA DEGLI ELABORATORI

INDICE

Rappresentazione di informazioni.....	4
Tabella Esadecimale	4
Come scrivere un numero in binario	5
Rappresentare un numero intero con segno	5
Numeri Decimali.....	5
Componenti di un Computer.....	7
Architettura MIPS e Linguaggio Macchina	8
R-Type.....	8
I-Type	9
J-Type.....	10
Tipi di Salto	10
Indirizzamento della Memoria.....	10
Reti Sequenziali e Combinatorie.....	11
Decoder	12
Multiplexer	12
PLA (Programmable Logic Arrays).....	13
Dispositivi ROM (Read Only Memory).....	13
Carry In e Carry Out.....	14
Segnale di Clock	14
SR Latch	14
D Latch	15
D Flip-Flop	16
Register File	16
DRAM e SRAM.....	17
Processo di Compilazione	18
Il Linker.....	18

Il Loader	19
Inizio seconda parte di Architettura.....	19
Assembly – Procedure.....	19
Implementazione Datapath	22
Datapath.....	22
Load Word	26
Store Word.....	27
Fetch Update PC	27
Salto Condizionato	27
Implementazione Singolo Ciclo.....	28
Implementazione Multiciclo.....	30
Eccezioni.....	34
Situazioni non normali	34
Convenzioni Di uso della memoria.....	35
Gestioni I/O.....	39
Memory Mapped I/O	39
Spazio Indirizzamento I/O	40
Interruzione programma	41
DMA (Direct Memory Access).....	41
Procedura Assembly 1.1.....	42
Procedure Innestate.....	42
Uso dello stack	43
Procedura Andata	44
Procedura Ritorno	44
Pipeline	45
Confronto fra le prestazioni di un processore a singolo ciclo e quelle di un processore con pipeline	46
Progettazione dell'insieme di istruzioni per architetture dotate di pipeline	48
Gli Hazard nelle pipeline	49

Hazard strutturali	49
Hazard sui dati.....	49
Hazard sul controllo.....	50
La Cache	51
Coerenza della Cache.....	52
Autori	53

Rappresentazione di informazioni

Come si fa a riconoscere se una sequenza è di tipo Int o Float?

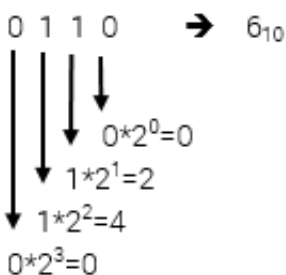
Sono le operazioni che permettono l'interpretazione di una sequenza di bit, per esempio ogni operazione può essere eseguita su variabili di un certo tipo.

Per rappresentare un tipo di informazione con precisione finita (numero finito di bit) sorgono diversi problemi:

- OVERFLOW: Se per caso si vuole rappresentare un numero troppo grande rispetto ai numeri rappresentabili per quel tipo di variabile
- RISULTATO: Facendo la divisione tra due numeri possiamo trovare dei numeri non rappresentabili
- PRECEDENZA: Se si svolgono operazioni senza rispettare l'ordine tra loro si giunge a risultati di rappresentazione errati

Tabella Esadecimale

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
										(10)	(11)	(12)	(13)	(14)	(15)



Come scrivere un numero in binario

$$213_{10} = ?_2$$

$$213-128=85$$

$$85-64=21$$

$$21-16=5$$

$$5-4=1$$

$$1-1=0$$

$$2^7 \quad 2^6 \quad 2^5 \quad 2^4 \quad 2^3 \quad 2^2 \quad 2^1 \quad 2^0$$

1 1 0 1 0 1 0 0

Le potenze di 2 utilizzate nella sottrazione equivalgono agli 1 mentre quelle non utilizzate sono gli 0.

Rappresentare un numero intero con segno

- **Modulo e Segno:** Con n bit si rappresentano i valori in: $(-2^{(n-1)}+1; 2^{(n-1)}-1)$
- **Complemento a 1:** Se prendiamo per esempio il numero

○ 10: 0 0 0 0 1 0 1 0 → 1 1 1 1 0 1 0 1

Il numero negativo di un numero in complemento a uno si ottiene cambiando ciascun bit, da 0 a 1 e da 1 a 0, dato che il complemento di x sarà $2^n - x - 1$.

- Complemento a 2:** Dati "n" bit numero positivo N è rappresentato in modo standard. Mentre $-N$ si rappresenta come $(2^n - N)$
 Prendo il numero in binario, inverte gli zeri con gli uno e gli uno con gli zeri, sommo un uno alla fine.

ESEMPIO: 01101010 $\rightarrow$ 10010101 +

10010110

- **Eccesso k:** Prendo il numero N, sommo k e lo trasformo in binario.

Numeri Decimali

- **Rappresentazioni in Virgola Fissa:** Divido l'intervallo per 1000, l'errore al massimo è di un millesimo. L'errore di approssimazione è costante su tutto l'intervallo; più piccolo è il numero, più importante è

l'errore. L'errore cresce con il crescere del valore.

Dati n bit, un bit lo uso per il segno; k bit per la parte intera, n-k-1 per la parte decimale.

Esempio:

dato un range di valori da 26,4 a 40, trovare se posso rappresentare il numero 30,913

$$26,4 \rightarrow 40$$

per prima cosa sottraggo la parte massima alla minima

$$40 - 26,4 = 13,6$$

ora, trovo il massimo delle combinazioni possibili con 4 bit, visto che il segno non mi interessa non serve che risparmi un bit per esso

$$2^4 = 16$$

divido il range di valori per il numero di combinazioni possibili

$$\frac{13,6}{16} = 0,85$$

Ho una precisione del 0,85, quindi ho un errore di 0,003 perché posso rappresentare solo due cifre dopo la virgola.

- **Rappresentazione in Virgola Mobile:** Il numero in virgola fissa ha un'unica cifra dopo la virgola.

Un numero si dice *Normalizzato* se, in virgola mobile, non possiede zeri iniziali (es: $1,10010 \cdot 2^2$).

Ci sono 3 tipi di rappresentazione in virgola mobile:

1) **Precisione Singola (32 bit);**

2) **Precisione Doppia (64 bit);**

3) **Precisione Estesa (80 bit);**

La Precisione singola formata da 32 bit è così suddivisa:

MSB (1bit)	Esponente (8bit)	Mantissa(23bit)
MSB (1bit)	Esponente (11bit)	Mantissa(52bit)

MSB = Most Significant bit : Identifica il segno del numero;

Mantissa = Tutto ciò che sta dopo la virgola, se si aumenta la mantissa si aumenta l'accuratezza del numero;

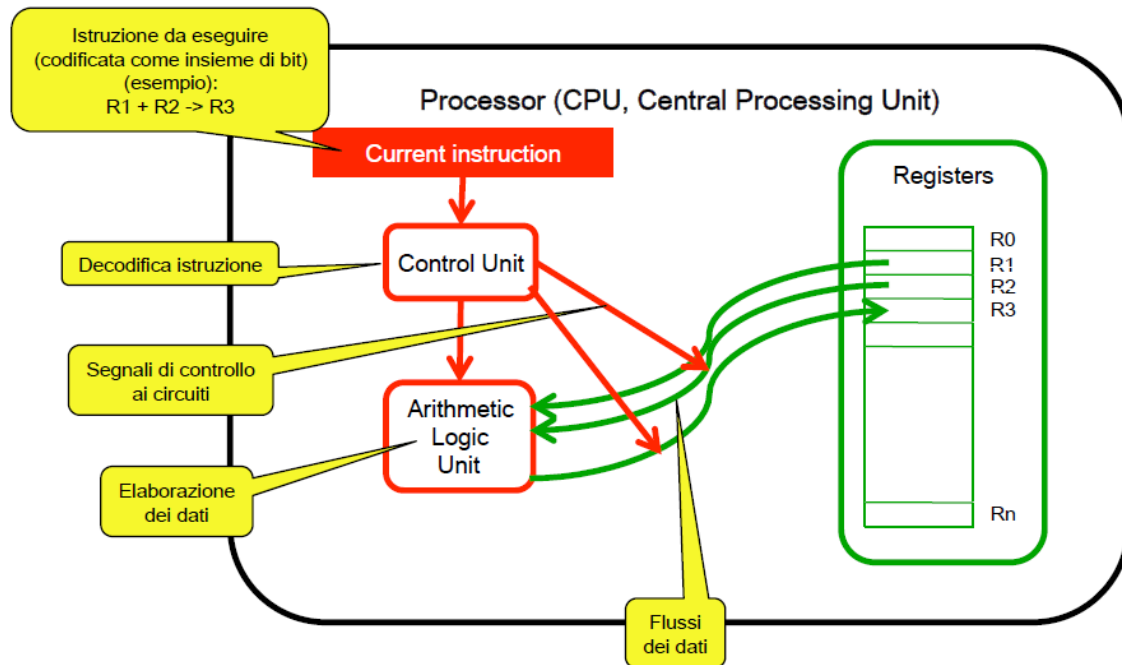
Esponente = Se si aumenta la dimensione dell'esponente, si aumenta l'intervallo di numeri rappresentabili;

Durante la generazione di un numero è possibile che si riscontrino alcuni problemi:

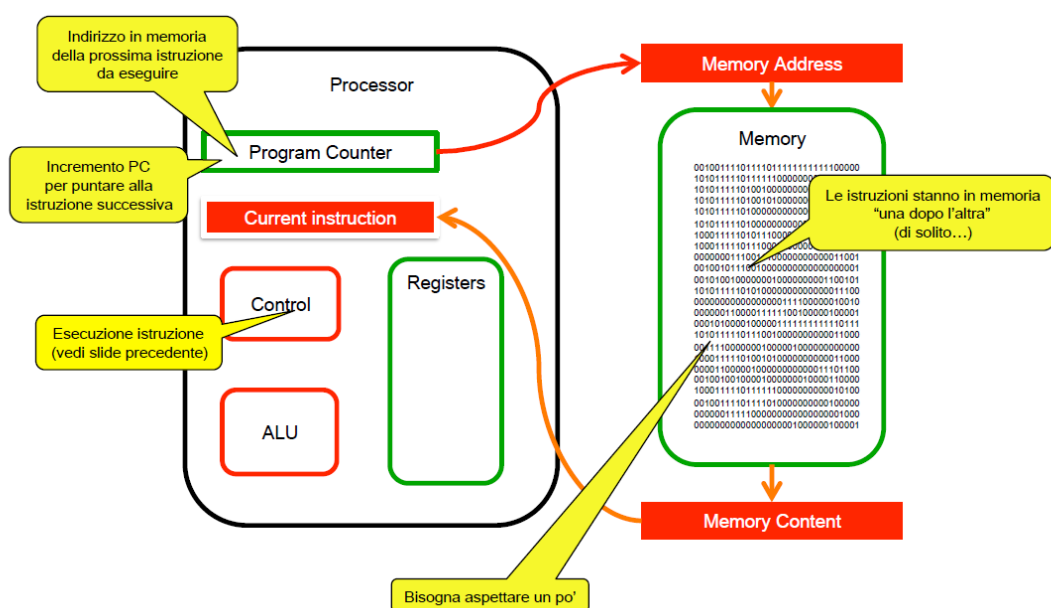
- **Overflow:** L'esponente positivo è troppo grande per poter essere rappresentato nel campo esponente;

- **Underflow:** L'esponente negativo è troppo grande per poter essere rappresentato nel campo esponente;
- **IEEE 754:** Rende implicito il primo bit dei numeri binari normalizzati che sappiamo essere uguale a 1;

Componenti di un Computer



Il processore riceve le istruzioni e i dati dalla memoria; l'Unità di ingresso scrive i dati nella memoria e l'unità di uscita legge i dati dalla memoria. L'Unità di controllo invia i segnali che determinano le operazioni che devono svolgere l'unità di elaborazione, la memoria e le unità di ingresso e uscita.



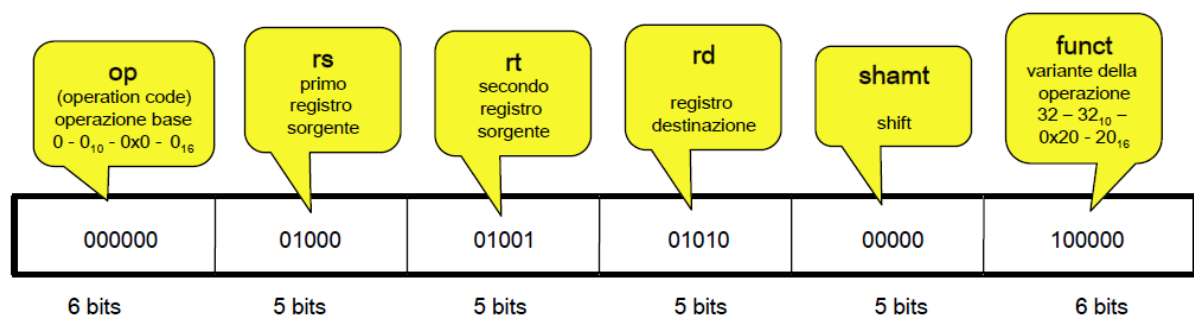
- **Register File:** Un elemento dello stato che contiene un insieme di registri che possono essere letti o scritti fornendo il numero del registro sul quale si vuole operare.
- **Memoria:** E' il luogo dove sono contenuti i programmi quando sono in esecuzione e i dati richiesti dai programmi stessi.
- **Memoria di Massa:** Chiamata anche memoria secondaria, è una memoria non volatile, utilizzata per conservare i programmi e i dati fra un'esecuzione e l'altra. Nei calcolatori attuali principalmente costituita dai dischi magnetici.
- **Alu:** L'arithmetic Logic Unit recupera i dati dai registri del processore, processa i dati nell'accumulatore (registro per svolgere la maggior parte delle operazioni aritmetico-logiche) e provvede a salvare il risultato nel registro di uscita. Altre strutture provvedono a salvare i dati dai registri alla memoria principale. L'Alu svolge le operazioni aritmetiche e logiche.
- **Program Counter:** E' un registro della CPU la cui funzione è quella di conservare l'indirizzo di memoria della prossima istruzione (in linguaggio macchina) da eseguire. E' un registro puntatore cioè punta a un dato che si trova in memoria all'indirizzo corrispondente al valore contenuto nel registro stesso. Il Program Counter è utilizzato in ciclo *fetch-execute* che costituisce la dinamica fondamentale nel funzionamento di un computer; tale ciclo è una ripetizione infinita dei seguenti passi:
 1. caricamento dell'istruzione riferita dal Program Counter;
 2. aggiornamento del PC in modo che contenga l'indirizzo dell'istruzione successiva;
 3. esecuzione dell'istruzione caricata.

Architettura MIPS e Linguaggio Macchina

Ci sono 3 tipi di istruzioni:

R-Type

E' un tipo di istruzione così strutturata:



E' formata da un OP (Operation Code) che identifica l'operazione, Il registro Sorgente, Il secondo registro Sorgente, Il registro Destinazione, lo Shift che di solito è sempre zero, e la Function che serve per sapere che tipo di fuzione si sta utilizzando.

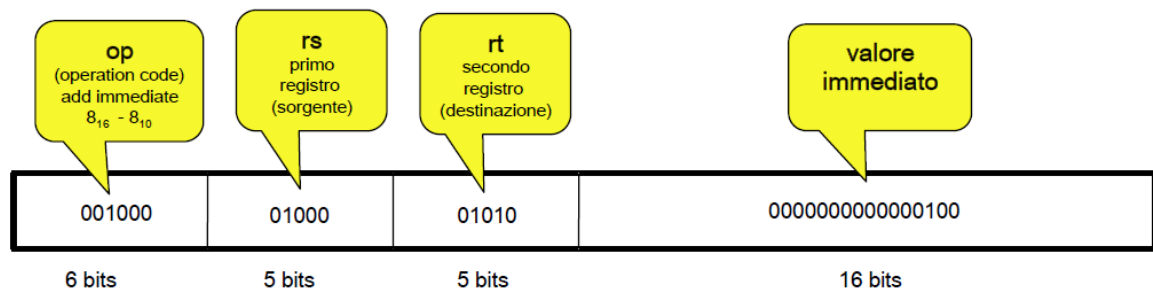
Solitamente le R-Type sono di tipo **add**, **sub**, **mul**, **div**, **and**, **or**.

Esempio: **add \$10, \$8, \$9** → sommo il contenuto del registro 8 e il contenuto del registro 9 e lo metto nel registro 10.

I-Type

E' un tipo di istruzione così strutturata:

“somma il valore 4 al contenuto del registro 8 e metti il risultato nel registro 10”



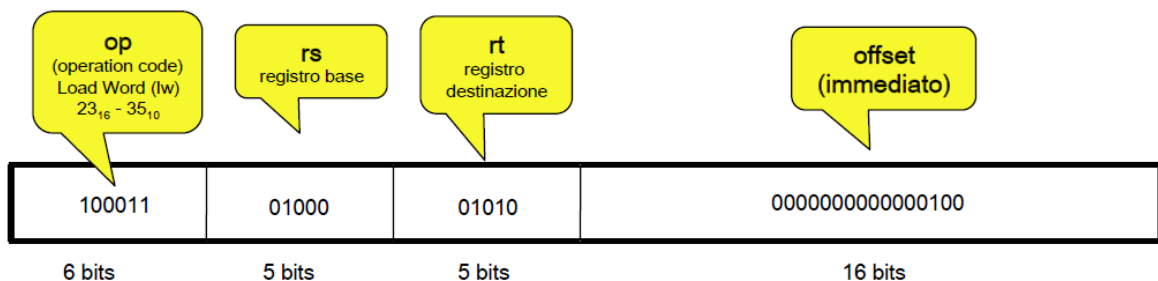
E' formata da un OP, il registro Sorgente, il registro Destinazione, e l'offset scritto in 16 bit.

Solitamente le I-Type sono di tipo **addi**, **load word**, **store word**.

Esempio: **addi \$10, \$8, 4** → sommo il valore 4 al contenuto del registro 8 e metto il risultato nel registro 10.

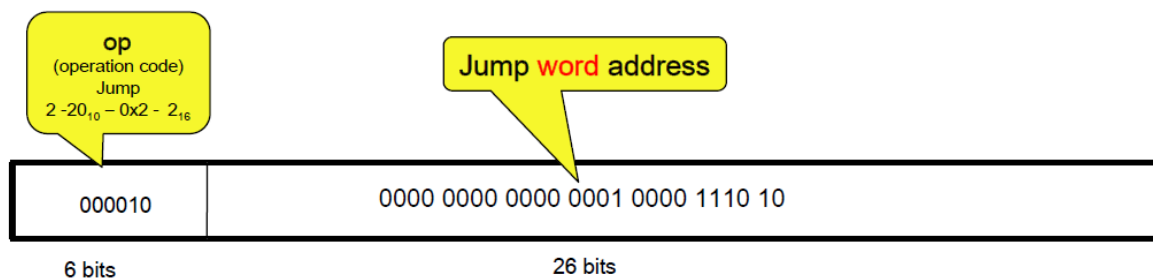
Esempio di Load Word:

“carica nel registro 10 il contenuto della parola (32 bit) che è all'indirizzo di memoria ottenuto come somma del registro 8 e dell'offset immediato 4”



J-Type

E' un tipo di istruzione utilizzata per i salti

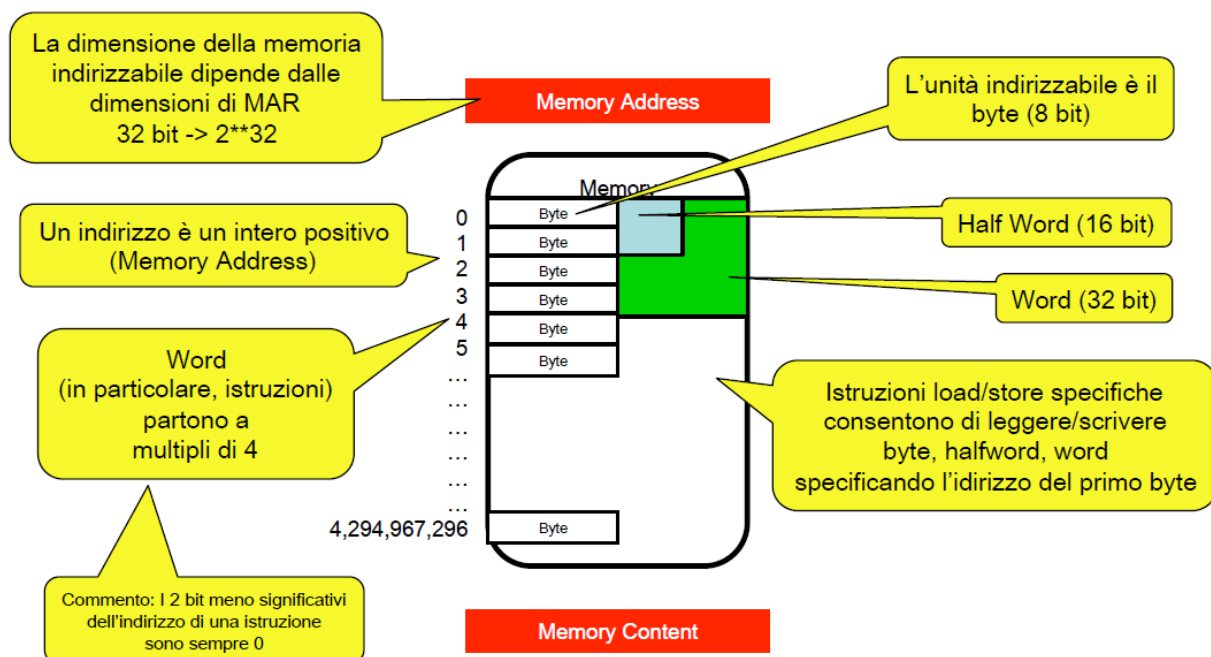


E' formata semplicemente da un Operation Code che determina il tipo di salto da fare e l'indirizzo a cui saltare.

Tipi di Salto

- **Incondizionato:** E' un salto che viene sempre eseguito senza il controllo di alcuna condizione.
L'operando destinazione contiene l'indirizzo dell'istruzione a cui il processore salta.
- **Condizionato:** E' un salto che viene eseguito o no dal processore a seconda di una precisa condizione.

Indirizzamento della Memoria

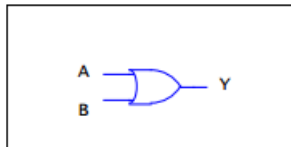


Reti Sequenziali e Combinatorie

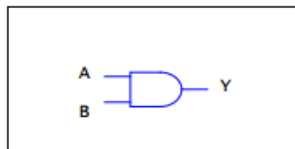
Le principali parti elettroniche dei computer sono costituite da circuiti digitali che, come è noto, elaborano segnali logici basati sullo 0 e sull'1.

Ora verranno descritte le principali porte logiche:

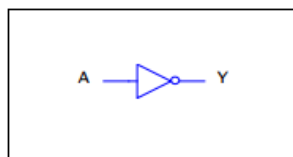
- **Somma Logica OR:** Si effettua su due o più variabili, l'uscita assume lo stato logico 1 se almeno una variabile di ingresso è allo stato logico 1.

	A	B	Y
	0	0	0
	0	1	1
	1	0	1
	1	1	1

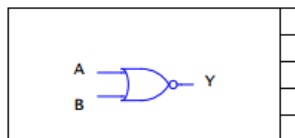
- **Prodotto Logico AND:** Si effettua su due o più variabili, l'uscita assume lo stato logico 1 solo se tutte le variabili di ingresso sono allo stato logico 1.

	A	B	Y
	0	0	0
	0	1	0
	1	0	0
	1	1	1

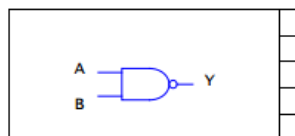
- **Negazione:** Si effettua su una sola variabile. L'uscita assume il valore logico opposto a quello applicato in ingresso.

	A	Y
	0	1
	1	0

- **Somma Logica Negata NOR:** Si effettua su due o più variabili, l'uscita assume lo stato logico 0 se almeno una variabile di ingresso è allo stato logico 1. In tutti gli altri casi $Y=1$. Corrisponde ad una OR con in cascata una NOT.

	A	B	Y
	0	0	1
	0	1	0
	1	0	0
	1	1	0

- **Prodotto Logico Negato NAND:** Si effettua su due o più variabili, l'uscita assume lo stato logico 0 se tutte le variabili di ingresso sono allo stato logico 1. In tutti gli altri casi $Y=1$. Corrisponde ad una AND con in cascata una NOT.

	A	B	Y
	0	0	1
	0	1	1
	1	0	1
	1	1	0

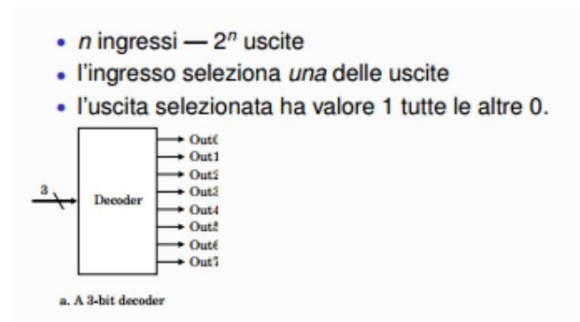
Collegando opportunamente le porte logiche si potranno definire tutti i tipi di possibili circuiti digitali.

- **Circuiti Combinatori:** le uscite del circuito dipendono esclusivamente dalle entrate (decoder, multiplexer..).
- **Circuiti Sequenziali:** le uscite del circuito dipendono sia dalle entrate che dallo stato interno del circuito (Latch, Flip Flop, Memorie..).

Decoder

Un circuito logico che viene frequentemente utilizzato nella costruzione di circuiti complessi è

il **decodificatore** (o **decoder**). Il tipo più diffuso di decodificatore ha un ingresso di n bit e 2^n (2 elevato alla n) uscite, in cui viene affermata una sola uscita per ciascuna combinazione di ingressi. Il decodificatore traduce l'ingresso su n bit in un segnale che corrisponde al valore binario dell'ingresso: se il valore dell'ingresso è pari ad i , allora l'uscita- i sarà vera e tutte le altre uscite saranno false.



Multiplexer

La sua uscita è uguale ad uno degli ingressi, scelto mediante un segnale di controllo.

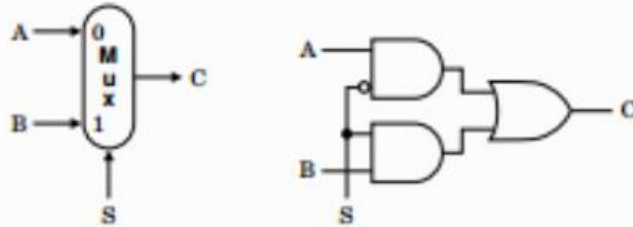
È possibile creare dei multiplexer con un numero arbitrario di ingressi di dato. Quando vi sono due ingressi di dato, vi è un solo segnale di selezione che identifica uno dei due ingressi. Se vi fossero n ingressi di dato, servirebbero $\lceil \log_2(n) \rceil$ ingressi di selezione. In tal caso il multiplexer sarebbe composto da tre parti fondamentali:

- Un decodificatore che genera n segnali, ciascuno corrispondente ad un diverso valore dell'ingresso di selezione.
- Una schiera di n porte and, ciascuna delle quali combina uno degli ingressi con uno dei segnali provenienti dal decodificatore.
- Un'unica grande porta or che raccoglie tutte le uscite delle porte and.

Due tipi di ingressi:

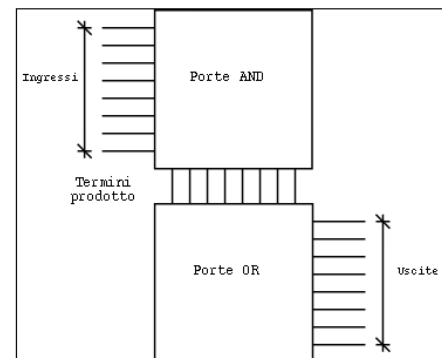
- n ingressi di controllo — 2^n ingressi segnale
- un'unica uscita;

il controllo seleziona quale segnale d'ingresso mandare in uscita



PLA (Programmable Logic Arrays)

Una PLA dispone di un insieme di ingressi e dei relativi valori complementati (che si possono ottenere per mezzo di invertitori) e di due stadi di logica. Il primo stadio è una matrice di porte and che formano un insieme di termini prodotto; ciascun termine prodotto combina le variabili di ingresso o i loro complementi. Il secondo stadio è una matrice di porte or, ciascuna delle quali calcola la somma logica di un numero arbitrario di termini prodotto.



Dispositivi ROM (Read Only Memory)

Una ROM è detta "memoria" poiché dispone di un insieme di locazioni che possono essere lette; tuttavia il contenuto di tali locazioni è fisso, determinato al momento della creazione della ROM stessa. Vi sono anche ROM programmabili (dette **PROM**) che vengono programmate elettronicamente non appena il progettista ne conosce il contenuto. Esistono inoltre delle PROM cancellabili (dette **EPROM**), che richiedono però un processo di cancellazione molto lento mediante luce ultravioletta, per cui vengono usate come memorie di sola lettura, eccezion fatta per la fase di sviluppo e per quella di correzione degli errori.

I dispositivi ROM sono dotati di un insieme di linee di indirizzo in ingresso e di un insieme di uscite: se la ROM contiene 2^n elementi indirizzabili (l'**altezza** della ROM), vi sono n linee di ingresso. Il numero di bit presenti in ciascun elemento indirizzabile determina invece il numero di bit di uscite ed è talvolta detto **ampiezza** della ROM. Il numero totale di bit in una ROM è pari al prodotto tra altezza ed ampiezza; questi due parametri sono indicati complessivamente come la **forma** della ROM.

Carry In e Carry Out

A	B	C	Riporto
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

→ $A+B=C$

A	B	Ci	D	Riporto
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

→ $A+B+Ci=D$

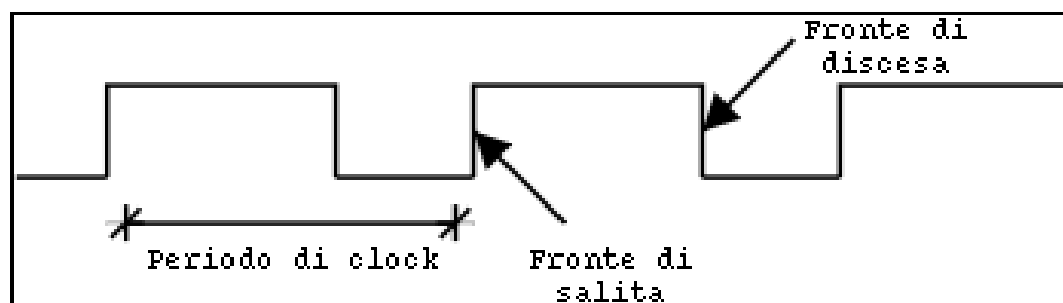
Carry_in rappresenta il riporto dell'operazione precedente.

Carry_out è il riporto dell'operazione appena eseguita e diventa il carry in della prossima operazione.

Segnale di Clock

Nella logica di tipo sequenziale i segnali di **clock** sono necessari per determinare quando debbono essere aggiornati gli elementi che contengono informazioni di stato. Un segnale di clock non è altro che un segnale che evolve indipendentemente con un **periodo** predeterminato e costante; la **frequenza di clock** è l'inverso del periodo.

Il **tempo di ciclo** o **periodo del clock** è diviso in due parti: la parte in cui il segnale di clock è alto e quello in cui è basso. Nella metodologia di temporizzazione di tipo **sensibile ai fronti (edge-sensitive)** tutte le variazioni di stato avvengono in corrispondenza di un fronte (**edge**) del segnale di clock.

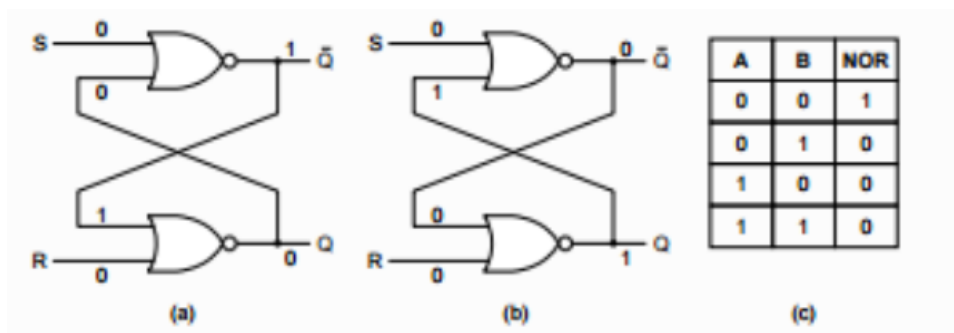


SR Latch

E' il latch più semplice dove S e R stanno per Set e Reset. Questo tipo di latch è composto da due porte NAND o da due porte NOR con collegamenti incrociati. Il bit immagazzinato è portato all'uscita Q e al suo complemento di uscita $\bar{Q}$.



S	R	Funzione
0	0	Latch (Memorizzazione)
0	1	Reset: $Q = 0$, $!Q = 1$
1	0	Set: $Q = 1$, $!Q = 0$
1	1	Non è ammesso



D Latch

Un latch di tipo D ha due ingressi e due uscite: gli ingressi sono il valore da memorizzare (chiamato D) ed il segnale di clock (chiamato C) che indica quando il latch deve leggere il valore sull'ingresso D e memorizzarlo. Le uscite sono semplicemente il valore dello stato interno (Q) ed il suo complemento ($\bar{Q}$). Quando l'ingresso di clock C è affermato, si dice che il latch è **aperto** ed il valore dell'uscita Q diviene il valore dell'ingresso D. Quando il clock C passa ad un valore non affermato, si dice che il latch è **chiuso** ed il valore dell'uscita Q rimane fissato all'ultimo valore che era stato memorizzato quando il latch era aperto. Poichè quando il latch è aperto il valore di Q varia ogni volta che D varia, questa struttura è spesso detta **latch trasparente**.

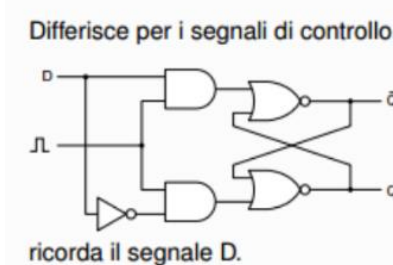
D Flip-Flop

Si può costruire un flip flop in modo che venga attivato o sul fronte di salita (fronte positivo) o su quello di discesa (negativo) del clock. Nei flip flop di tipo D, l'uscita viene memorizzata in corrispondenza del fronte del clock. Poiché l'ingresso D è campionato ad ogni fronte del clock, deve rimanere valido in un opportuno intervallo di tempo immediatamente prima ed immediatamente dopo il fronte del clock. Il tempo minimo per cui l'ingresso deve rimanere valido prima del fronte del clock è detto **tempo di set-up** (tempo di preparazione), mentre il tempo minimo per cui deve rimanere valido dopo il fronte del clock è detto **tempo di hold** (tempo di mantenimento).

Register File

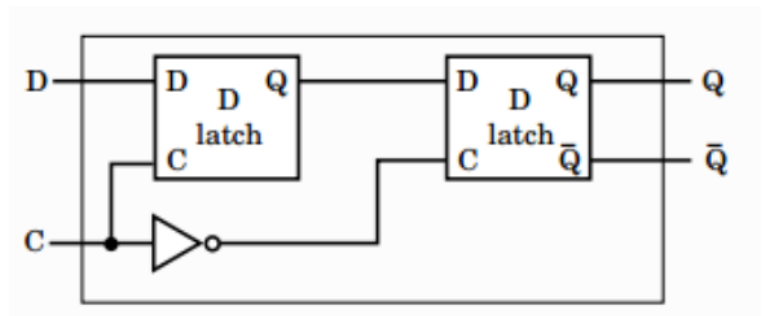
La struttura intorno alla quale è costruita l'unità di elaborazione è il **register file**. È formato da un insieme di registri che possono essere letti o scritti fornendo il numero del registro a cui fare accesso. Un register file può essere realizzato mediante un decodificatore per ciascuna porta di lettura o scrittura ed una matrice di registri

costruiti a partire da flip flop di tipo non ne modifica lo stato, occorre del registro ed in uscita si otterrà nel registro. Per una scrittura in un ingressi: il numero di registro, il clock che controlli l'operazione di possono implementare con due



ampiezza pari al numero di bit nel register file. L'implementazione della porta di scrittura è più complessa poichè è permesso modificare solo il contenuto del registro designato: lo si può fare usando un decodificatore per generare un segnale in grado di determinare in quale registro scrivere.

Che cosa avviene se lo stesso registro viene letto e scritto nello stesso ciclo di clock? Poichè la scrittura del registro avviene in corrispondenza del fronte del clock, il registro sarà valido durante la sua lettura. Il valore



D. Poichè la lettura di un registro solo fornire in ingresso il numero semplicemente il dato contenuto registro sono invece richiesti tre dato da scrivere ed un segnale di scrittura. Le porte in lettura si multiplexer, ciascuno dei quali di

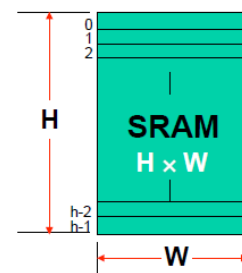
letto sarà il valore scritto in un periodo di clock precedente. Se si volesse leggere il valore attualmente in corso di scrittura, sarebbe necessaria della logica aggiuntiva nel register file o al suo esterno.

- **Read Reg #1** : Numero del primo registro da leggere
- **Read Reg #2** : Numero del secondo registro da leggere
- **Read Data 1** : Valore del primo registro letto sulla base del *ReadReg#1*
- **Read Data 2** : Valore del secondo registro letto sulla base del *ReadReg#2*
- **Write Reg#** : Numero del registro su cui scrivere
- **Write Data** : Valore del registro da scrivere in base a *WriteReg#*

DRAM e SRAM

La **SRAM** (Static RAM) è più veloce

- Per la sua realizzazione vengono usati dei latch
- È usata per realizzare memorie veloci, come le memorie cache
- Tempi di accesso (2008) intorno a 0,5 – 2,5 ns
- SRAM a basso consumo (5-10 volte più lente)



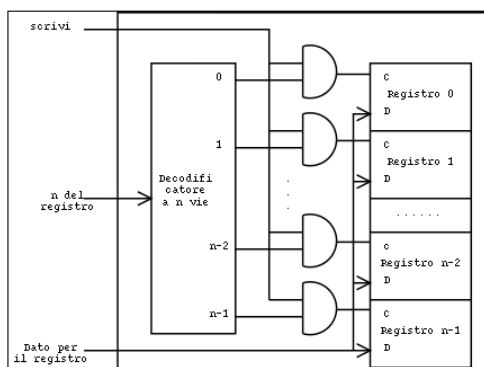
Numero di bit dell'indirizzo: $\log_2 H$

L'**SRAM** è realizzata come matrice di latch HxW con:

- Larghezza o ampiezza W (numero di latch per ogni cella)
- Altezza H (numero di celle indirizzabili)
- Singolo indirizzo per lettura o scrittura
- Non è possibile scrivere e leggere contemporaneamente, a differenza del Register File

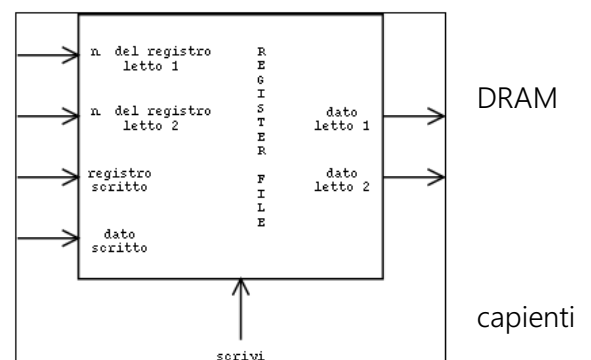
La **DRAM** (Dynamic RAM) è più capiente ma più lenta

- Tempi di accesso (2008) intorno a 50-70 ns
- Non è realizzata tramite latch
- Ogni bit è memorizzato tramite un condensatore



principale

- È necessario "rinfrescare" il contenuto delle a intervalli di tempo prefissati
- È usata per realizzare memorie come quella

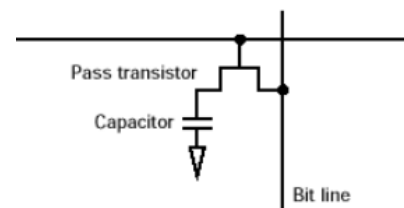
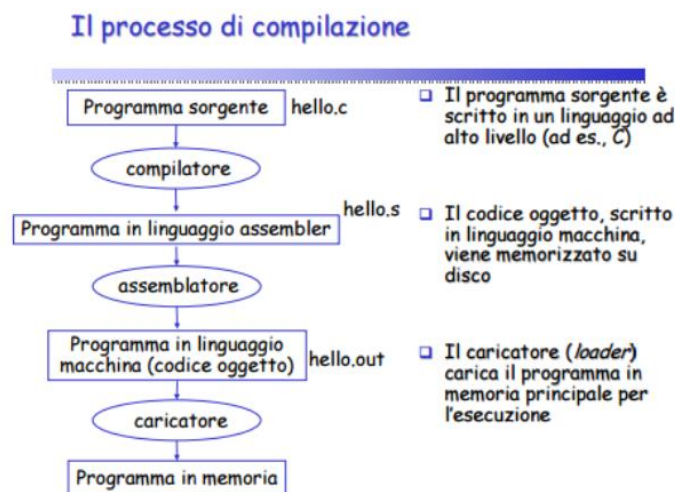


La DRAM è meno costosa, perché è realizzata con un solo transistor per bit, e un condensatore. Il condensatore possiede la carica (0/1), il transistor viene chiuso, trasferendo il potenziale elettrico del condensatore sulla Bit Line (output), grazie al segnale affermato della Word Line; la specifica Word Line è attivata sulla base dell'indirizzo di memoria richiesto.

I condensatori mantengono memorizzati solo per alcuni ms. Necessario il *refresh* dinamico delle DRAM, effettuati leggendo, e subito riscrivendo i valori appena letti. Il *refresh* avviene ad intervalli fissi, occupa circa il 2% del tempo totale, e avviene per righe.

Processo di Compilazione

- Nella prima fase, il programma ad alto livello viene tradotto nel linguaggio assembler utilizzando un apposito programma detto compilatore.
- Dopo la fase di compilazione, il programma scritto nel linguaggio assembler viene tradotto in linguaggio macchina utilizzando un apposito programma detto assembler (assembler).
- Spesso con il termine compilazione l'intero processo di traduzione da linguaggio ad alto livello linguaggio macchina (essendo l'assembler spesso integrato con il compilatore).



Il Linker

Il linker (link editor o collegatore) ha il compito di collegare tra loro vari moduli che compongono lo stesso programma

- Programma sorgente suddiviso in più file che vengono compilati separatamente creando diversi file oggetto
- Utilizzo di funzioni di libreria

Il linker collega tra loro i file contenenti il codice oggetto dei vari moduli che costituiscono il programma, unendovi anche il codice delle funzioni di libreria utilizzate, producendo un file contenente il codice eseguibile, memorizzato su disco.

Compiti del linker:

- Porre simbolicamente in memoria istruzioni e dati
- Determinare gli indirizzi dei dati e delle etichette dei salti
- Unire i riferimenti interni ed esterni alle procedure

Il linker produce un file oggetto che può essere eseguito sulla macchina: stesso formato del file oggetto ma con tutti i riferimenti risolti e senza informazioni di rilocazione.

Il Loader

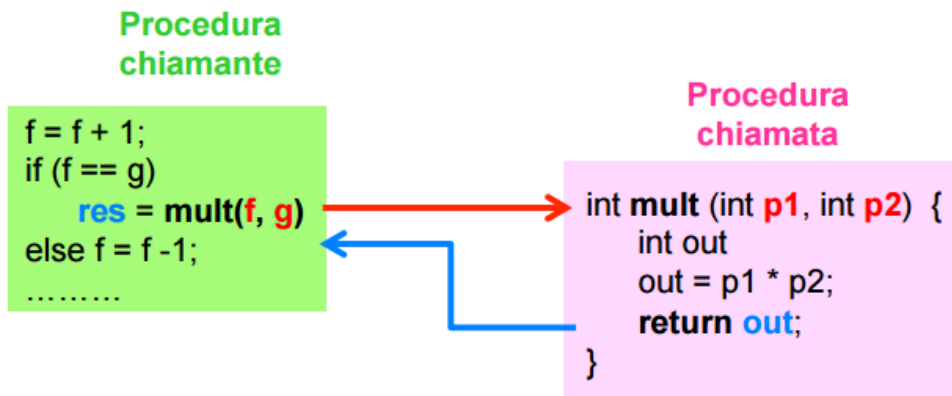
Una volta che il linker ha creato il file eseguibile, esso viene di solito memorizzato su disco. All'atto dell'esecuzione, il S.O. la carica dal disco e ne avvia l'esecuzione.

Il loader si occupa di far partire il programma:

- Leggere l'intestazione per determinare la dimensione dei segmenti testo e dati
- Crea uno spazio di memoria sufficiente per testo e dati
- Copia le istruzioni e i dati in memoria
- Copia nello stack i parametri (se presenti) passati al main.
- Inizializza i registri e lo stack pointer.
- Salta ad una procedura di inizializzazione che copia i parametri nei registri appostiti dallo stack e poi invoca la procedura di inizio del programma (main() in c).

Inizio seconda parte di Architettura

Assembly – Procedure



❖ Cosa succede?

La procedura **chiamante** deve predisporre i parametri di ingresso in un posto accessibile alla procedura invocata e trasferire il controllo alla procedura **chiamata**.

Mentre, la procedura **chiamata** deve: allocare lo spazio di memoria necessario all'esecuzione ed alla memorizzazione dei dati (con il *record di attivazione*), eseguire il compito richiesto, memorizzare il risultato in un luogo accessibile alla procedura **chiamante** e restituire il controllo alla procedura **chiamante**.

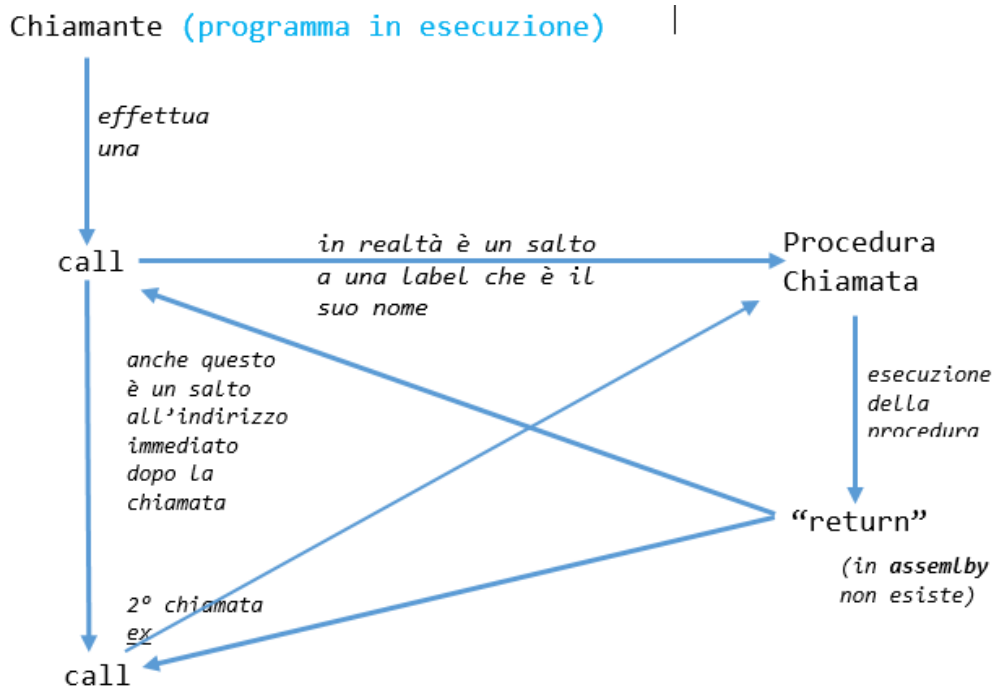
La procedura **chiamata** necessita un'istruzione apposita che : **cambi il flusso** di esecuzione (cioè salti alla procedura) e **salvi l'indirizzo di ritorno** cioè all'istruzione successiva alla chiamata a procedura **chiamante**.

L'istruzione che permette questi compiti è la **jal (jump and link)**

[SINTASSI: **jal** <indirizzo_procedura>]

Esso salta all'indirizzo con etichetta <indirizzo_procedura> memorizzando il **valore corrente del Program Counter** in \$ra. (l'indirizzo dell'istruzione successiva: \$(Program_Counter)+4).

La **procedura chiamata**, come **ultima istruzione**, esegue: **jr \$ra** per effettuare il salto all'indirizzo di ritorno della procedura.



Passaggio di parametri

Ci sono due convenzioni utilizzate per gestire il passaggio di parametri e risultati.

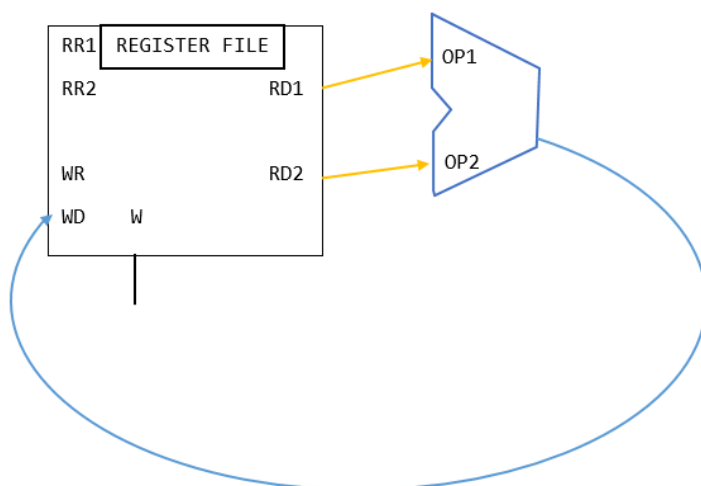
\$a0 - \$a3 → registri argomento: sono quelli utilizzati per il passaggio di parametri; [input]

\$v0 - \$v1 → registri valore: sono quelli utilizzati per la restituzione dei risultati; [output]

Dal punto di vista hardware sono registri come tutti gli altri, ma il loro utilizzo è per il passaggio dei parametri, è una convenzione programmatica che deve essere rispettata per consentire di scrivere procedure che:

- Possono essere scritte senza bisogno di sapere come è fatto il programma che le chiama
- Possono essere chiamate senza bisogno di sapere come sono fatte al loro interno

Un parametro può essere un dato o un indirizzo.



N.B. un parametro può essere un *dato* o un *indirizzo* !

la ≠ lw

I passaggi di parametri possono essere per valore (**lw**) o per indirizzo (**la**).

```

.data
num1: .word 50
num2: .word 30
num3: .word 20
num4: .word 40
result 1.....
result 2.....

.text

lw $a0, num1      → passaggio di parametri
lw $a1, num2      → passaggio di parametri

jal somma1        → chiamata alla procedura in cui si salva in
$ra il return address e salta alla
sw $v0, result1   procedura. SW: → memorizzazione del primo
risultato

lw $a0, num3
lw $a1, num4
jal somma1
sw $v0, result2

somma1:           → inizio della procedura
add $v0, $a0, $a1
jr $ra            → indirizzo di ritorno

```

Implementazione Datapath

Datapath

Un dato viene memorizzato solo sul fronte di *discesa* del **clock**.

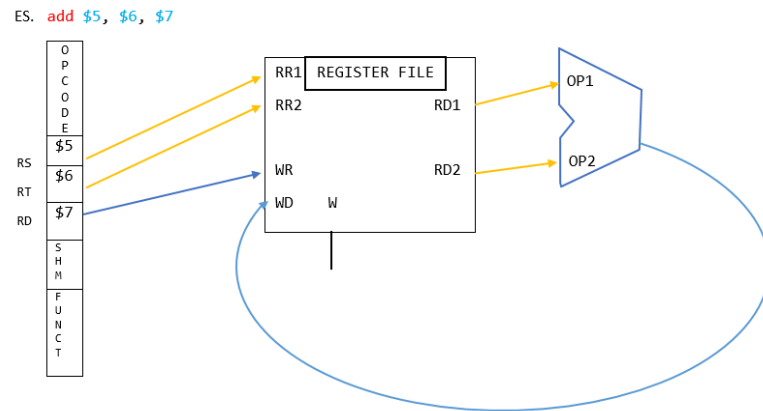
Esso è un insieme di *unità di calcolo* come ad esempio le unità di elaborazione (**ALU**), i *registri* e i moltiplicatori necessari per l'esecuzione delle istruzioni nella **CPU**.

Il passaggio di due operandi attraverso la ALU e la memorizzazione del risultato in un nuovo registro viene detto *ciclo di data path*. Tale ciclo definisce ciò che è in grado di fare una **macchina**; più veloce è il ciclo del *datapath*, più veloce sarà la macchina. Ogni istruzione ISA (*assembly*) viene eseguita in uno o più cicli del *data path*; diversi cicli sono necessari per istruzioni complesse come la divisione, per esempio.

In architetture non parallele, il ciclo di *data path* corrisponde al ciclo di **clock**. La velocità con cui viene compiuto un ciclo di *data path* contribuisce significativamente a determinare la velocità della CPU.

Il *data path*, insieme all'**unità di controllo**, costituisce il nucleo stesso della CPU (**core**). Altri componenti che completano la CPU sono: la *memoria*, l'*unità di ingresso input* e l'*unità di uscita*.

Di default si effettuano operazione di lettura sul *register file*, se si vuole scrivere si deve abilitare l'ingresso "Write" del *register file*, che quando viene memorizzato ci sarà un cambiamento di stato.



Il *datapath* rappresenta il "giro" dei dati all'interno di un elaboratore.

Esso può essere a ciclo singolo (*single cycle*) o multi ciclo (*multicycle*).

Nel funzionamento a ciclo singolo i singoli componenti vengono sfruttati una ed una sola volta per compiere le operazione e, di conseguenza, per eseguire operazioni più complesse è necessario l'utilizzo di più componenti.

Nel funzionamento del multi-ciclo, invece, è possibile ridurre il numero di componenti dell'elaboratore utilizzando un segnale di temporizzazione del *Clock*.

In ogni "giro" di *clock* è possibile utilizzare tutti i componenti del circuito, ma dato che essi mantengono lo stato alla fine dell'elaborazione è possibile riutilizzare questi componenti, partendo dallo stato finale dell'elaborazione precedente, ad ogni nuovo ciclo di *clock*.

Per la composizione di un *datapath* si terrà conto di un sottoinsieme delle possibili operazioni MIPS, che include:

- **Load Word** (carica una *word* in memoria)
 - `lw rt, address`
 - Sorgente: `address = [rs] + offset`
 - Destinazione: `rt`
- **Store Word** (salva una *word* in memoria)
 - `sw rt, address`
 - Sorgente: `rt`
 - Destinazione: `address = [rs] + offset`
- **Add-Sub-And-Or** (confronti e operazioni matematiche/logiche)
 - `<nome> rd,rs,rt`
 - Sorgente: `rs, rt`

- Destinazione: **rd**
- [N.B. Queste operazioni si differenziano in base all'OPCode: **0x20 add 0x22 sub 0x24 and 0x25 or**]
- **SetOnLessThan** (imposta **rd = 1**, solo se **rs < rt**, altrimenti **rd = 0**)
 - **slt rd, rs, rt**
 - Sorgente: **rs, rt**
 - Destinazione: **PC = PC + offset(Label)+4**
- **Jump** (salta incondizionatamente ad una **Label** o **indirizzo**)
 - **j Label(addr)**
 - Sorgente: **<null>**
 - Destinazione: **PC = PC[31-28] OR offset*4**

Inoltre si utilizza un segnale di *clock* molto semplice, composto da un'onda quadra con una sola fase attiva. Questo causa un ritardo sulla tempistica delle operazioni, in quanto nel primo ciclo di clock viene eseguito il *fetch* dell'istruzione (**lettura**) mentre sul secondo giro vengono eseguite le operazioni dipendenti dall'OPCode.

Il *datapath* si compone di alcuni elementi fondamentali:

- **Unità di memoria**

Unità addetta alla memorizzazione di dati ed istruzioni. L'indirizzo di un'istruzione è salvato in un registro denominato Program Counter (PC)

- **Register file**

Registri indicizzati con un numero che permettono il salvataggio e la lettura dei dati nell'elaborazione di un programma

- **Estensione con segno**

Unità che si occupa di trasformare un dato senza segno a **16 bit** in un dato con segno a **32 bit**, utilizza soprattutto dalle istruzioni di salto (**branch** e **jump**), e delle istruzioni che necessitano di un **offset** di indirizzo (**load word** e **store word**)

- **Shift a sinistra (2bit)**

Unità che si occupa di "shiftare" a sinistra il dato di 2 bit; ES: 10001 → SHIFT → 1000100

Necessario nelle operazioni di salto (**branch** e **jump**), per calcolare correttamente il target address.

- **ALU**

Necessaria per incrementare il PC per indirizzare correttamente l'istruzione successiva e per eseguire le operazioni di calcolo sui registri.

Essa deve implementare nativamente le funzioni di **AND**, **OR**, **ADD** e **SUB** con **overflow**, **SLT** e una funzione di confronto.

Dato che alcune delle unità sopra citate sono utilizzate per scopi diversi, una su tutte l'abilità della ALU di incrementare il PC e di eseguire operazioni sui dati dei registri, è necessario inserire dei multiplexer all'interno del datapath per poter selezionare correttamente la sorgente e la destinazione corretta dei dati/indirizzi di memoria.

Un multiplexer andrà inserito a valle del PC e collegato all'output del PC stesso e della ALU per consentire di scegliere se recuperare la memoria all'indirizzo specifico per le istruzioni oppure quello per i dati.

Altri due multiplexer sono necessari a monte della ALU, per consentire di scegliere l'input corretto tra un registro e il PC in un caso, e tra un secondo registro, l'output dello shifter(per i salti), e il valore costante 4(per l'incremento del PC).

Esistono inoltre degli altri segnali, detti segnali di controllo (control signals) che permettono la corretta attivazione e funzionalità dei componenti, ad esempio il PC, la memoria e il registro istruzioni necessitano di un segnale per la loro scrittura, la memoria necessita di un segnale per la lettura, alla ALU serve un insieme di segnali per la scelta dell'operazione da compiere, in base all'OPCode, ed ad ogni multiplexer serve una o più linee di controllo per scegliere la corretta linea da abilitare.

Le istruzioni di salto rappresentano un caso particolare di istruzione, e per la loro esecuzione si rendono necessarie altre linee di controllo tra la ALU e il PC.

Con le jump e le branch infatti, esistono 3 valori possibili da scrivere nel PC dopo l'esecuzione degli eventuali controlli:

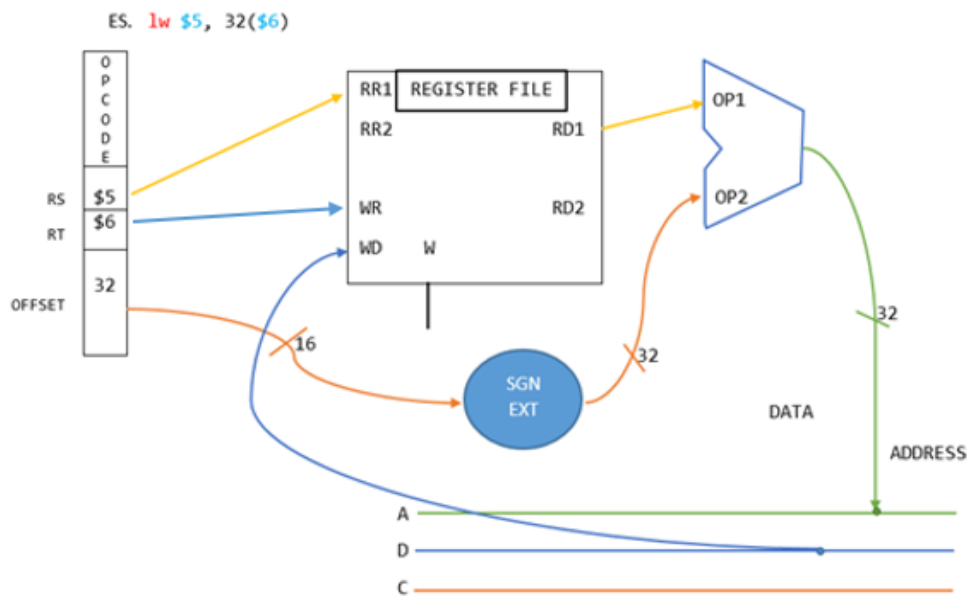
- $PC + 4$ nel caso di una branch che non rispetti le condizioni attese
- Uscita dell'ALU nel caso di branch che rispetti le condizioni, quindi $PC = PC + target$
- $PC = PC[31:28] || \text{indirizzo di jump}$ nel caso di jump incondizionato verso un indirizzo specifico

Quindi, si determinano 3 casi possibili di azione per incrementare il PC:

- Nel caso di jump (salto incondizionato), il nuovo valore è scritto direttamente all'interno del PC
- Se il valore contenuto nei due registri è uguale, quindi la **beq rs,rt** corrisponde a vero, il valore del PC viene sovrascritto con quello in uscita dalla ALU
- Viene incrementato il valore di PC di 4, come una normale continuazione di programma.

Per questo motivo si rendono necessari due ulteriori segnali di scrittura verso il PC, uno per la scrittura condizionata (*BEQ* vera o meno), ed uno per la scrittura condizionata (Jump). Semplificando, troviamo due segnali in OR fra loro, per verificare il salto incondizionato e l'eventuale salto condizionato, e quest'ultimo segnale frutto di una AND tra il salto condizionato ed il segnale di Zero della ALU (usato per verificare se i due registri rispettino la condizione di branch).

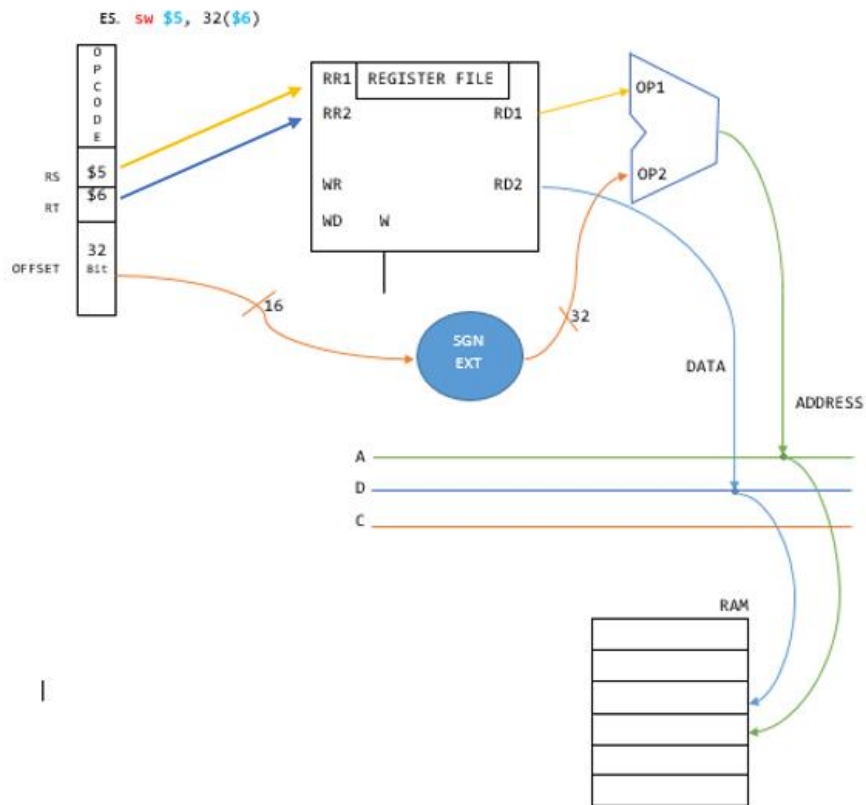
Load Word



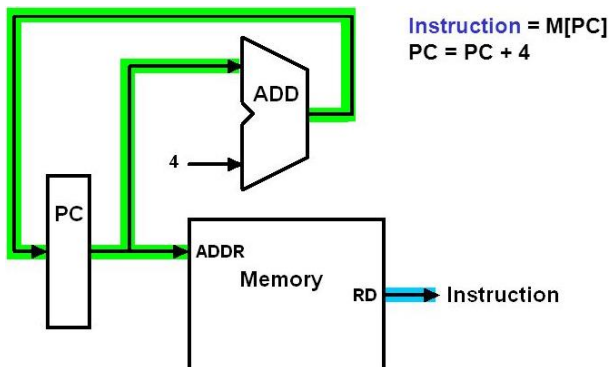
Risultato: l'indirizzo di memoria. Dalla CPU bisogna andare a leggere il contenuto della locazione di memoria dell'indirizzo ottenuto e restituisce il contenuto al register file. Per questa operazione si sfruttano gli address, data, control bus →

- Il risultato dell'ALU (che è un indirizzo) viene messo nell'address-bus;
- Si va a leggere in memoria il contenuto dell'indirizzo dell'address-bus e lo si mette nel data-bus;
- Ciò che c'è sul data-bus passa sul Write-Data del register file;
- Il control-bus esegue la funzione di controllo di questa operazione

Store Word



Fetch Update PC

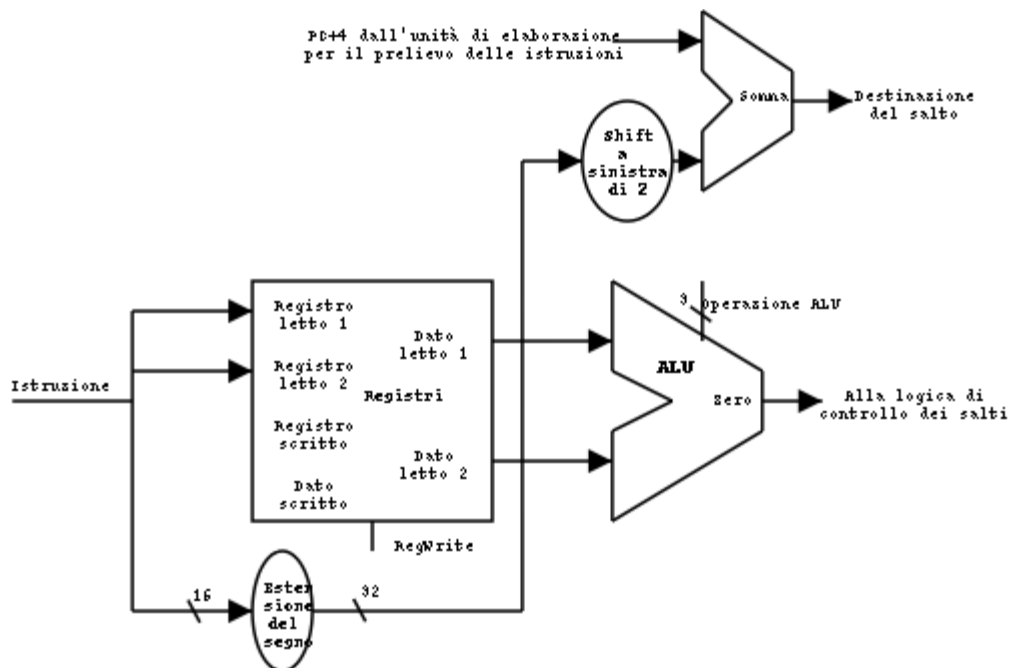


La *Fetch-Update-PC* si occupa solo di aggiornare il *Program Counter* (+4 per l'allineamento) alla prossima word per leggere il campo contenuto nell'indirizzo calcolato dall'ALU, ed eseguire l'istruzione.

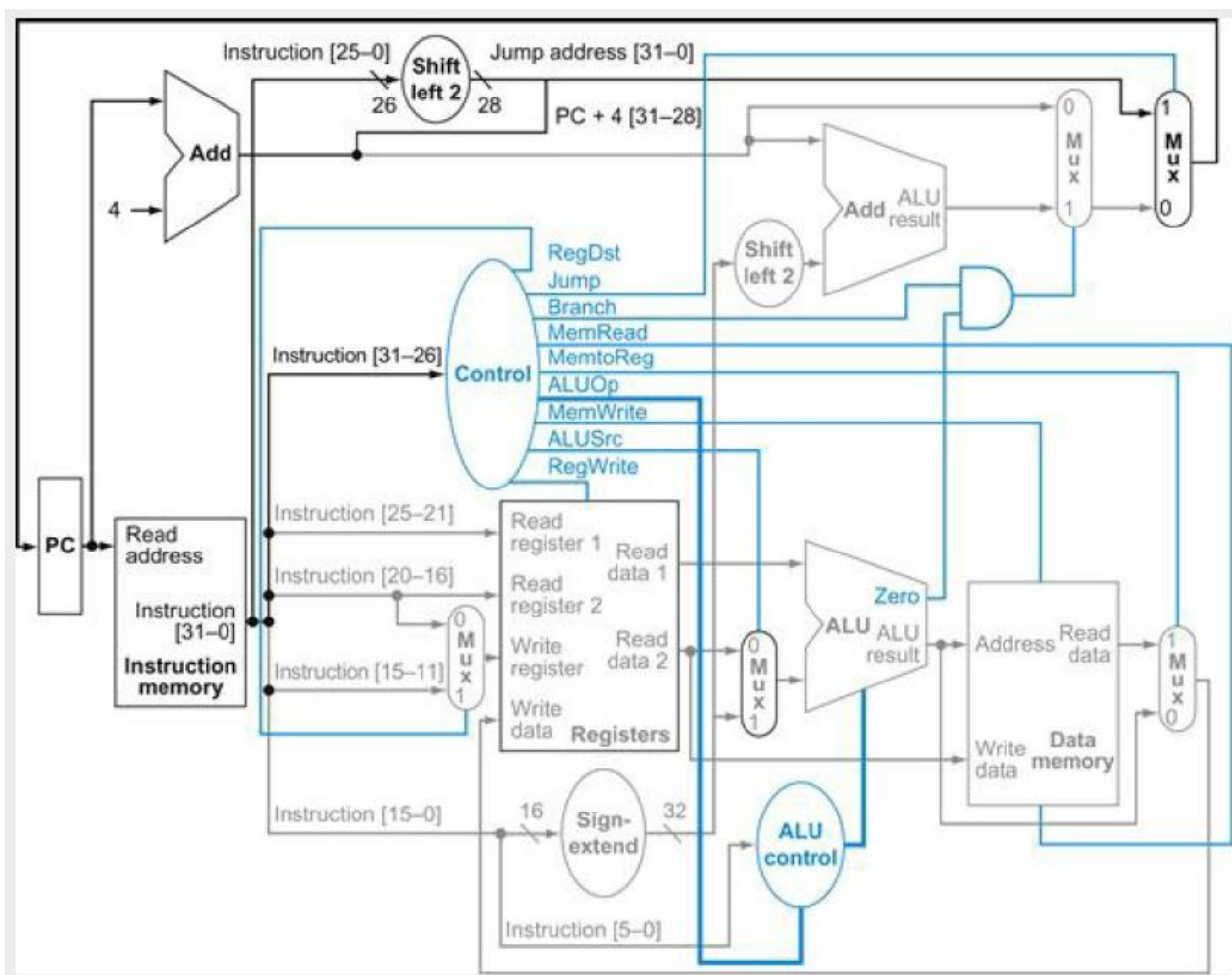
Salto Condizionato

Ogni salto condizionato è un I-Type e la condizione viene controllata dall'ALU, che esegue operazioni logico-aritmetiche.

Esempio: Per verificare la condizione di uguaglianza si fa la **differenza bit a bit**, se i due numeri sono uguali il risultato sarà un insieme di **zeri**. Ogni singolo risultato della differenza bit a bit viene messo in una **NOR** (se nella differenza abbiamo un insieme di zeri, portandoli nella OR rimane sempre zero, ma con la negazione diventa 1 e si abilita la scrittura; se abbiamo invece almeno un 1, la OR diventa 1 e che negata fa 0 e non viene abilitata la scrittura.)



Implementazione Singolo Ciclo



L'implementazione generica di un'istruzione:

- Usa il registro *Program Counter*, interno alla CPU, per fornire alla memoria l'indirizzo dell'istruzione

- Legge l'istruzione dalla memoria (**Fetch**)
- Interpreta i campi dell'istruzione per decidere esattamente cosa fare (**Decode**)
- Usa l'ALU per l'esecuzione (**Execute**)
 - **add/sub/and/or/slt** usano l'ALU per le operazioni corrispondenti, e il Register File per accedere ai registri
 - Le istruzioni di **lw/sw** usano l'ALU per calcolare gli indirizzi di memoria
 - L'istruzione di **beq** usa l'ALU per controllare l'uguaglianza dei registri
- Modifica il PC, e reintera il ciclo

Il controllo della CPU a singolo ciclo combinatorio.

Il datapath è invece un circuito sequenziale; i suoi output dipendono anche dal valore dei registri

L'indirizzo della memoria dati oppure il valore da immagazzinare in memoria in conseguenza di una store, dipendono dai valore dello Stato interno del Datapath (ovvero dai contenuti dei registri).

Si attende che tutti i circuiti siano stabili, sia quelli del datapath che quelli di controllo, prima di attivare il fronte di salita/discesa del clock.

Clock in AND con segnali di controllo di scrittura (registri / memoria)

I valori vengono scritti in corrispondenza del fronte di salita / discesa del clock solo se i segnali relativi sono affermati

Ciclo di Clock determinato sulla base del cammino più che i segnali elettrici devono attraversare

I circuiti del controllo agiscono in parallelo alla scrittura dei registri. Implementazione MultiCiclo

Implementazione Multiciclo

L'obiettivo nel decomporre l'esecuzione delle operazioni in cicli di clock dovrebbe essere quello di **bilanciare la quantità di lavoro** svolta in ciascun ciclo, in modo da minimizzare il periodo di clock. Si può iniziare suddividendo l'esecuzione di ciascuna istruzione in una serie di passi, ciascuno corrispondente ad un ciclo di clock, che siano pressappoco della stessa lunghezza.

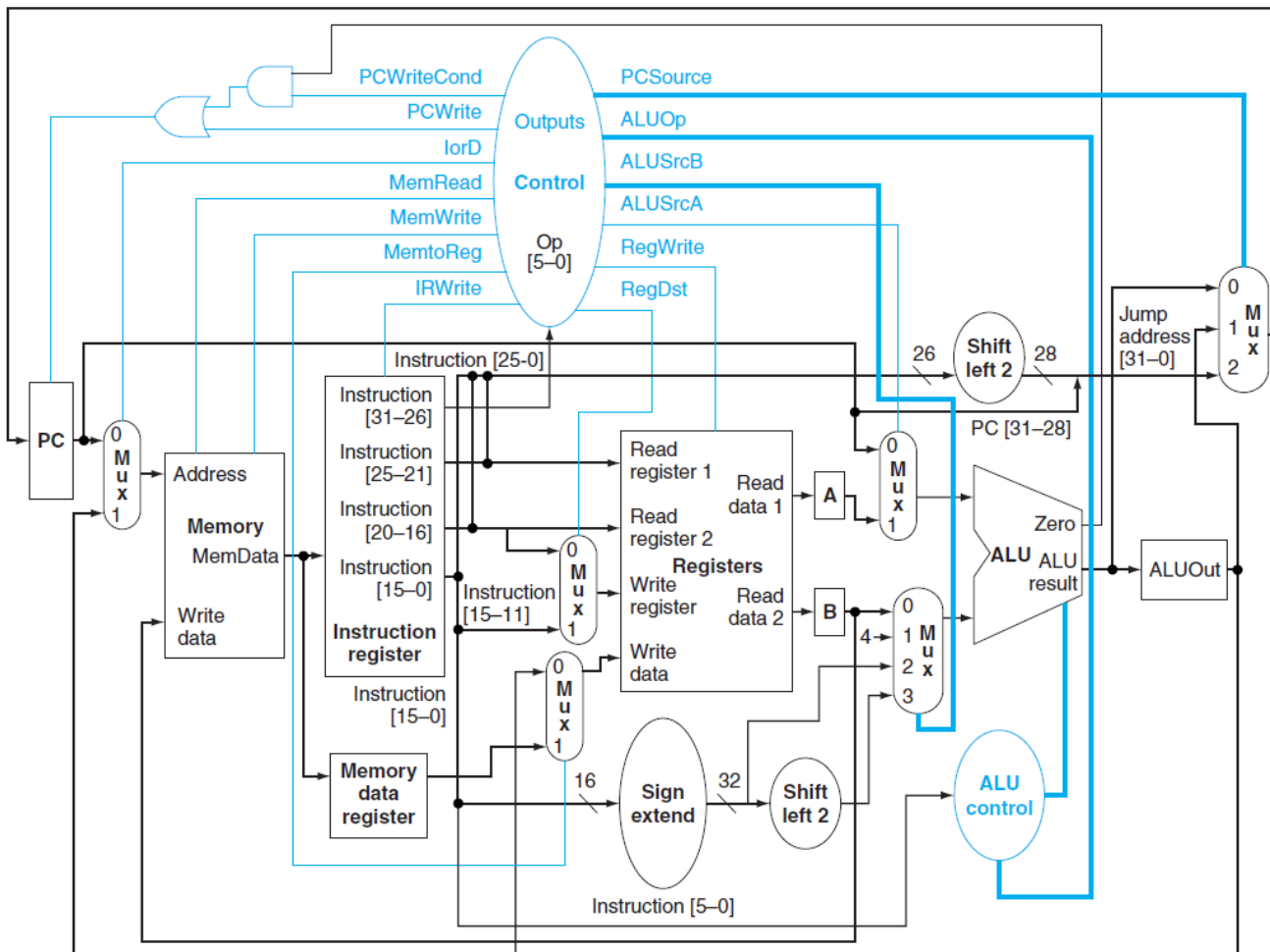
La stessa unità funzionale (*Memoria, Register File, PC*) può essere utilizzata più di una volta per istruzione, purchè in cicli di Clock diverso.

Le **principali caratteristiche** della versione *multi-ciclo* sono:

- È sufficiente *una sola unità di memoria* per istruzioni e dati.
- È presente **una sola ALU**, anzichè una ALU e due sommatori.
- **Un solo Multiplexer** che riceve **reg. A** e PC
- Un **multiplexer** che riceve **reg. B**, +4 (per raggiungere la prossima istruzione sul PC), **Sign Ext** (per portare a 32 bit l'immediate/offset), lo **Shift Left 2** (per allineare le branch) che è dopo il **Sign Ext**.

Uno o più registri sono aggiunti a ciascuna unità funzionale per memorizzarne l'uscita fino a quando, in un ciclo di clock successivo, il valore verrà utilizzato.

Ogni istruzione è divisa in fasi, a cui corrisponde un ciclo di Clock (Controllo sequenziale).



Necessità di memorizzare risultati intermedi in registri **temporanei**

- *Registri Temporanei*: non visibili al programmatore, salvano i dati prodotti in un ciclo di Clock che dovranno essere utilizzati dalla stessa istruzione in un ciclo successivo
- *Registri visibili al programmatore*: dati utilizzati dalle istruzioni successive

In un *solo* ciclo di clock si può effettuare una sola operazione tra queste: (Accesso in *Memoria*; Accesso ai *Registri* sia in *lettura* che in *scrittura*; Operazione ALU).

Alla fine di ogni ciclo, impiegando registri addizionali interni, si memorizzano i valori richiesti dei cicli successivi.

Per soddisfare i suddetti requisiti sono stati aggiunti i seguenti registri:

Il **registro dell'istruzione** (o **Instruction Register, *IR***) ed il **registro dei dati di memoria** (o **Memory Data Register, *MDR***) memorizzano l'uscita della memoria rispettivamente durante le operazioni di lettura di un'istruzione e di un dato. Sono utilizzati due registri separati in quanto i due valori saranno necessari nello stesso ciclo di clock.

I **registri A e B** memorizzano i valori dei registri letti dal *register file*.

Il **registro ALUOut** memorizza l'uscita della ALU.

Tutti i registri, fatta eccezione per *IR*, memorizzano i dati solamente tra un ciclo di clock e quello immediatamente successivo e pertanto non richiedono un segnale di controllo della scrittura.

Il registro *IR*, invece, deve memorizzare l'istruzione fino alla fine della sua esecuzione, richiedendo quindi un segnale di controllo per la scrittura.

Avendo sostituito le tre ALU dell'unità di elaborazione a singolo ciclo con una sola ALU l'unità di elaborazione è stata modificata come segue:

- Si è aggiunto un nuovo **Multiplexer** sul primo ingresso della **ALU**, che seleziona il **registro A** o **PC**
- Il **multiplexer** a due vie presente sul secondo ingresso della **ALU** è sostituito da uno a 4 vie
- Un ingresso aggiuntivo è la costante **4** per incrementare il **PC**
- L'altro ingresso è quello per estendere a **32 bit** il campo dell'**offset/immediate** (e allinearli alle **word** → **Shift Left** di 2)

Per i salti condizionati e incondizionati è necessario aggiungere dei supporti per le istruzioni di tipo **branch** e **jump**. Vi sono 3 possibili sorgenti per il valore da scrivere in **PC**:

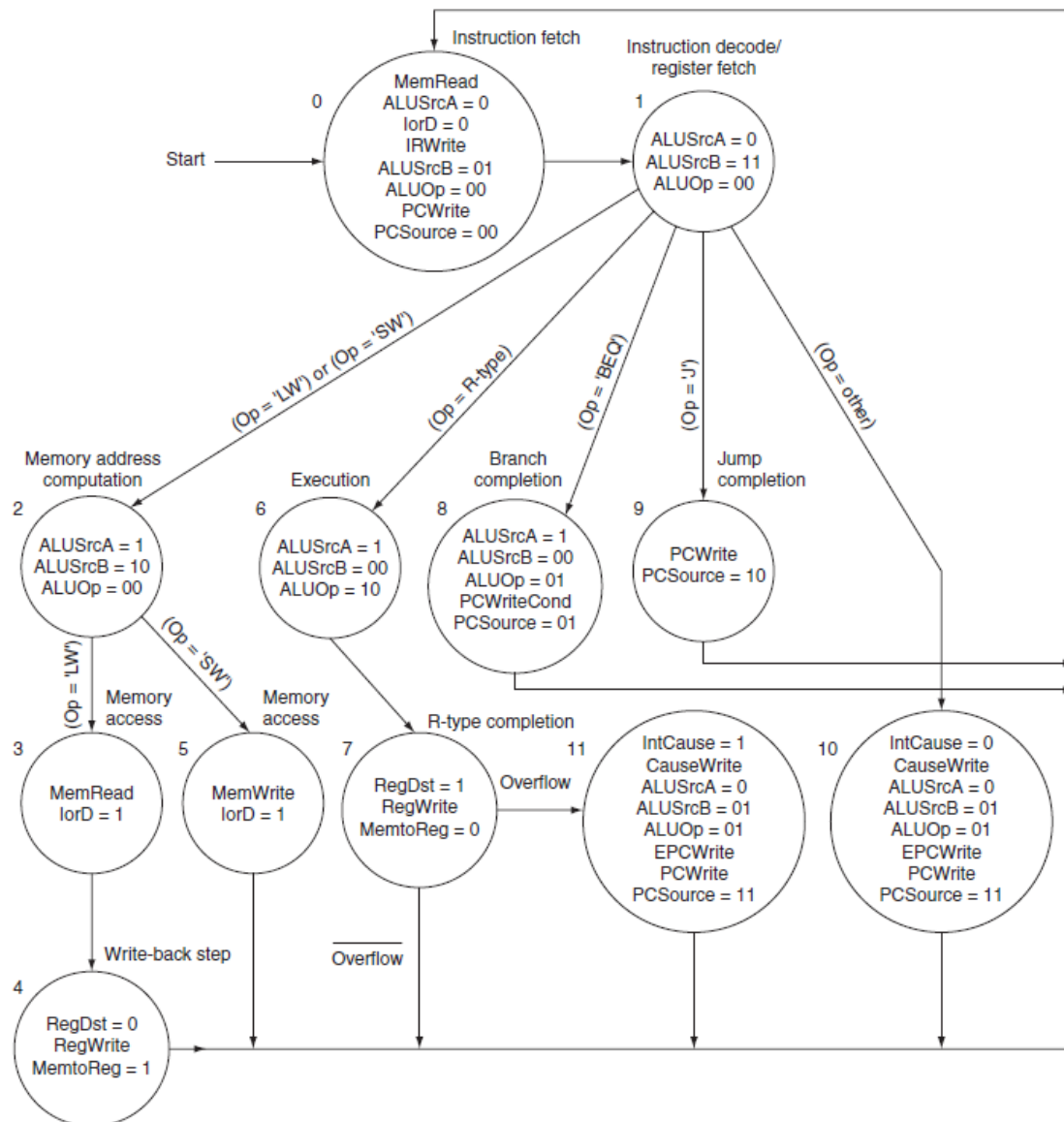
- L'uscita **dell'ALU** (**PC + 4**) durante il reperimento dell'istruzione; questo valore viene scritto direttamente dentro il **PC**

- Il registro **ALUOut** che conterrà a calcolo effettuato l'indirizzo del salto condizionato
- I 26 bit meno significativi dell'IR, scalati in due posizioni a sinistra e concatenati con i 4 bit superiori di PC incrementato da utilizzare con una jump

Azioni dei segnali di controllo di 1 bit		
Nome del segnale	Effetto quando non affermato	Effetto quando affermato
RegDst	Il numero del registro destinazione per Registro scritto proviene dal campo rt.	Il numero del registro destinazione per Registro scritto proviene dal campo rd.
RegWrite	Nessuno.	Nel registro specificato sull'ingresso Registro scritto è scritto il valore presente sull'ingresso Dato scritto.
ALUSrcA	Il primo operando della ALU è PC.	Il primo operando della ALU è il registro A.
MemRead	Nessuno.	Il contenuto della cella di memoria determinata dall'ingresso Indirizzo è posto sull'uscita MemData.
MemWrite	Nessuno.	Il contenuto della cella di memoria determinata dall'ingresso Indirizzo è sostituito dal valore presente sull'ingresso Dato scritto.
MemtoReg	Il valore inviato all'ingresso Dato scritto dei registri proviene da ALUOut.	Il valore inviato all'ingresso Dato scritto dei registri proviene da MDR.
lorD	L'indirizzo fornito alla memoria proviene da PC.	L'indirizzo fornito alla memoria proviene da ALUOut.
IRWrite	Nessuno.	L'uscita della memoria viene scritta in IR.
PCWrite	Nessuno.	PC viene scritto, la provenienza del valore è controllata da PCSource.
PCWriteCond	Nessuno.	PC viene scritto se anche l'uscita Zero della ALU è attiva.

Azioni dei segnali di controllo di 2 bit		
Nome del segnale	Valore	Effetto
ALUOp	00	La ALU calcola una somma.
	01	La ALU calcola una sottrazione.
	10	La ALU calcola l'operazione determinata dal campo funct dell'istruzione.
ALUSrcB	00	Il secondo ingresso della ALU proviene dal registro B.
	01	Il secondo ingresso della ALU è la costante 4.
	10	Il secondo ingresso della ALU è il valore dei 16 bit meno significativi di IR, estesi a 32 bit con segno.
	11	Il secondo ingresso della ALU è il valore dei 16 bit meno significativi di IR, estesi a 32 bit con segno e scalati a sinistra di 2 bit.
PCSource	00	In PC viene scritta l'uscita della ALU (PC+4).
	01	In PC viene scritto il contenuto di ALUOut (l'indirizzo di destinazione del salto condizionato).
	10	In PC viene scritto l'indirizzo di destinazione del salto incondizionato (IR[25-0] scalato a sinistra di 2 bit e concatenato con PC+4[31-28]).





Eccezioni

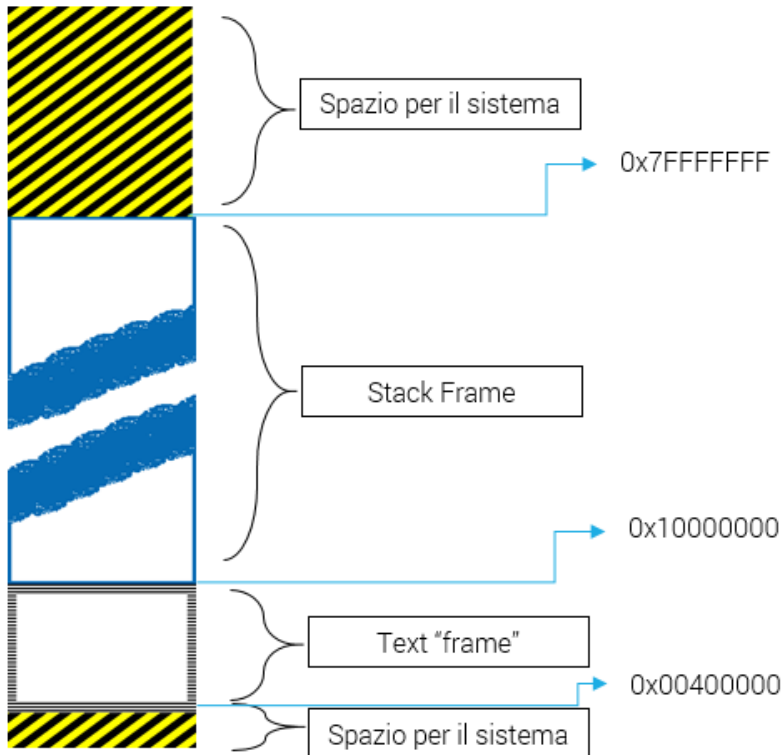
Situazioni non normali

Quando ci sono delle deviazioni nella sequenza di controllo/flusso pensato dal programmatore si classificano 3 tipi di eccezione:

- **Eccezioni vere** e proprie (not exist opcode, bad address, overflow), avvengono in modo sincrono, accadono in fase d'esecuzione dell'istruzione
- **Interruzioni**, esse portano ad alterazioni (derivanti da eventi esterni) del flusso d'esecuzione. Non sono previste dal programmatore. Sono asincrone e hanno luogo in un momento imprevisto dal flusso (es. I/O)

- **Traps**, eccezioni *sincrone* cioè, un'istruzione in cui la fase esecutiva è scatenare un'eccezione (ottenere l'esecuzione di dati servizi).

Convenzioni Di uso della memoria



MEMORIA 1

SYSCALL → "TRAPS" gestita inversamente, non si va nel gestore eccezioni, il sistema "esegue" le "cose" della syscall, come *modifiche hardware* per alterazione flusso, oppure non esegue il gestore.

Modifiche HARDWARE

EPC	Exception program counter
-----	---------------------------

E' un registro a 32bit utilizzato per memorizzare l'indirizzo dell'istruzione coinvolta

→ Attività hardware → macchina a stati.

CAUSE	E' un registro utilizzato per memorizzare la causa dell'eccezione;
-------	--

Gestore delle eccezioni:

I due tipi di eccezione generabili dall'implementazione dell'esempio sono l'esecuzione di un'istruzione non valida e l'overflow aritmetico; l'operazione fondamentale che il processore deve compiere al verificarsi di un'eccezione consiste nel salvare l'indirizzo dell'istruzione responsabile dell'eccezione nell'**Exception Program**

Counter (EPC) e trasferire il controllo al sistema operativo, ad un qualche indirizzo specifico.

Il sistema operativo potrà quindi prendere i provvedimenti più adeguati, che potrebbero consistere nel fornire qualche servizio al programma utente, eseguire qualche azione predeterminata in risposta all'overflow o terminare l'esecuzione del programma segnalando un errore. Dopo aver compiuto le azioni necessarie in risposta all'eccezione, il sistema operativo può terminare il programma o prenderne l'esecuzione utilizzando EPC per determinare il punto da cui ripartire.

Per poter gestire correttamente l'eccezione, il sistema operativo deve conoscerne il motivo, oltre all'istruzione che l'ha causata. Vi sono due metodi per rendere disponibile il motivo di un'eccezione: il metodo utilizzato nell'architettura MIPS è di includere un registro di causa (detto registro **Causa**) contenente un campo che indica il motivo dell'eccezione.

Un metodo alternativo prevede di adottare le **interruzioni vettorizzate**: nel corso di un'interruzione vettorizzata, l'indirizzo a cui trasferire il controllo è determinato dalla causa dell'eccezione. Ad esempio, per gestire i due tipi di eccezione sopra elencati, si potrebbero definire i seguenti indirizzi

Tipi di eccezione	Indirizzo del vettore di eccezione (esa)
Istruzione indefinita	C0 00 00 00 _{esa}
Overflow aritmetico	C0 00 00 20 _{esa}

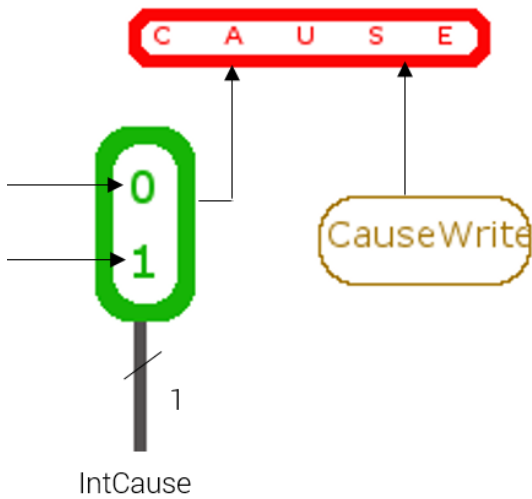
Il sistema operativo ricava il motivo dell'eccezione dall'indirizzo a cui viene attivato; gli indirizzi sono separati da 32byte, equivalenti ad 8 istruzioni, ed in questa breve sequenza il sistema operativo deve registrare il motivo dell'eccezione ed eseguire qualche semplice elaborazione. Quando l'eccezione non è vettorizzata, tutte le eccezioni utilizzano lo stesso punto iniziale ed il sistema operativo deve decodificare il registro di stato per conoscere la causa.

Si assuma di voler implementare il sistema di eccezioni utilizzato nell'architettura MIPS. Occorrerà aggiungere due registri all'unità di elaborazione

Saranno necessari due segnali di controllo aggiuntivi per pilotare la scrittura dei registri EPC e Cause, detti **EPCWrite** e **CausaWrite**, inoltre servirà un segnale di controllo di 1bit per fornire il valore corretto al bit meno significativo del registro Causa, chiamato **Causalnt**. Infine, occorrerà poter scrivere nel registro PC l'indirizzo iniziale della procedura di gestione delle eccezioni, per esempio C0 00 00 00_{esa}.

Attualmente PC è alimentato dall'uscita di un multiplexer a tre vie, controllato dal segnale PCSource: lo si può trasformare in un multiplexer a quattro vie, con l'ingresso aggiuntivo collegato al valore costante C0 00 00 00_{esar} ed un valore di PCSource pari ad 11_{due} selezionerà tale valore.

Siccome PC viene incrementato nel primo ciclo di ciascuna istruzione, non si può semplicemente scrivere il valore di PC in EPC. Si dovrà invece utilizzare la ALU per sottrarre quattro da PC e scrivere il risultato in EPC. La porta in ingresso di EPC è dunque connessa all'uscita della ALU



In MIPS sono 32bit. 5 bit servono per definire la causa dell'eccezione:

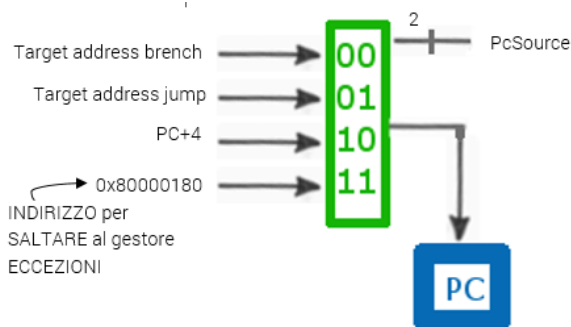
- **Registro causa** = 12 → istruzione indefinita
- **Registro causa** = 13 → 1 overflow aritmetico

I segnali di controllo sono:

- **CauseWrite** - scrittura nel registro Causa
- **Causalnt** - Dato per il registro Causa
- **PCsrc** - Aggiunta di un terzo input al PC per la scelta dell'indirizzo istruzione risposta alle eccezioni.

- Per mantenere i valori corretti nei registri
- SW in opportune aree di memoria

Salvo il valore dei registri per non mutarli a fine gestione (durante potrebbe servirgli) eccezione.

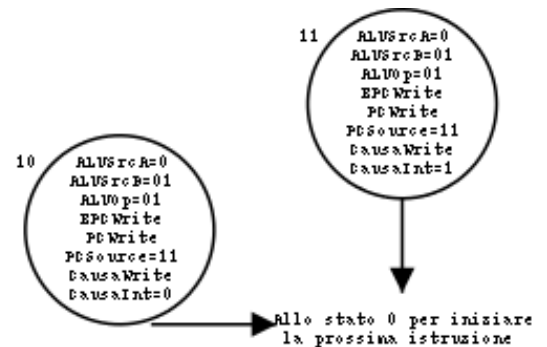


Con questa unità di elaborazione si potranno gestire i diversi tipi di eccezioni con uno stato ciascuno: in ciascun caso, lo stato fornisce il valore al registro Causa, ricalcola il valore originario di PC salvandolo in EPC e scrive in PC l'indirizzo della procedura di gestione delle eccezioni.

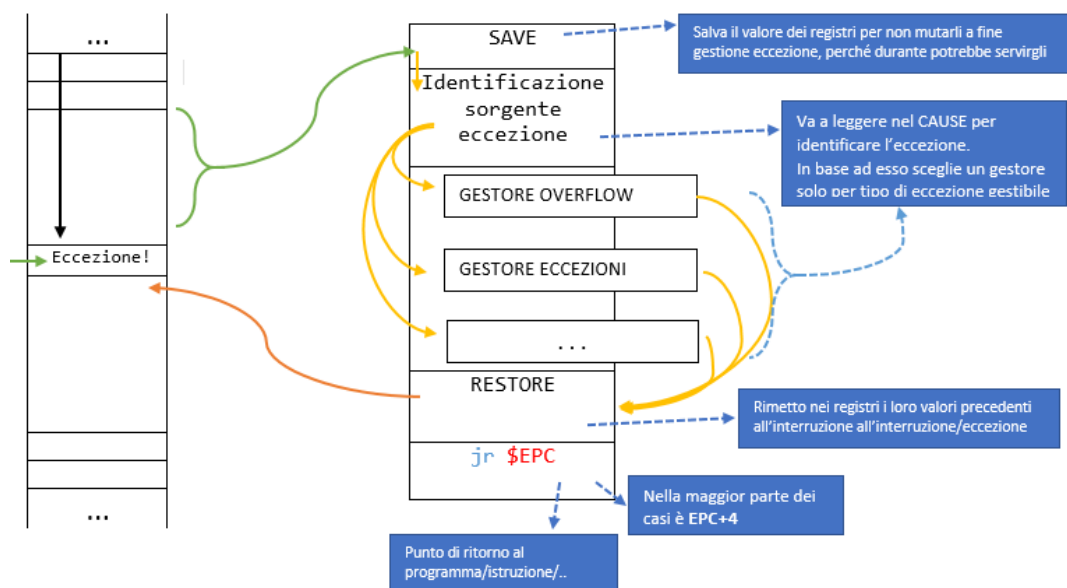
Per aggiungere le due eccezioni considerate saranno quindi sufficienti i seguenti due stati:

Le due eccezioni vengono rilevate in modo diverso:

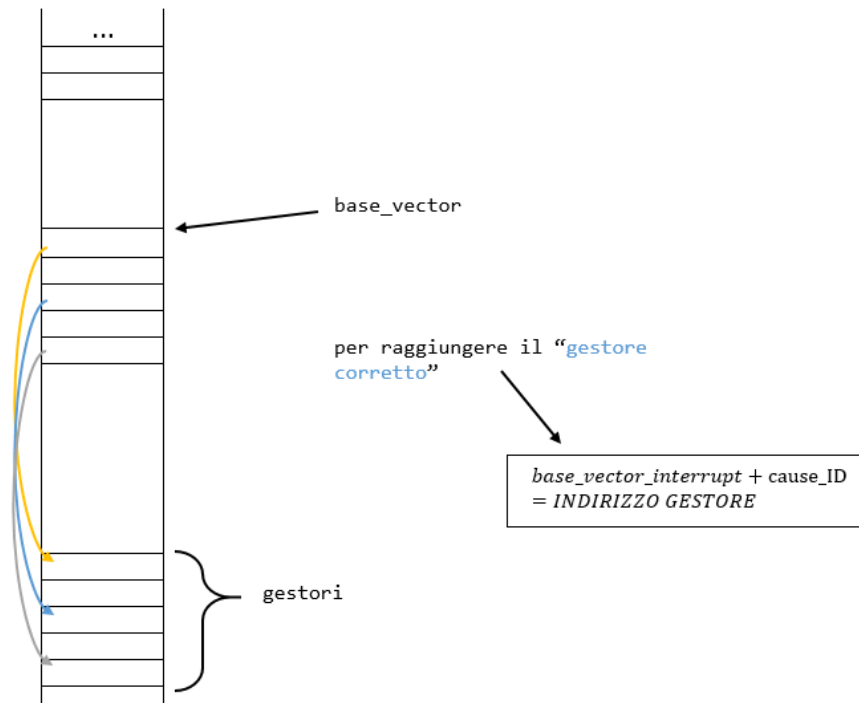
- **Istruzione indefinita:** viene rilevata quando, a causa del valore di op, non esiste uno stato futuro definito in uscita dallo stato 1. L'eccezione viene gestita definendo che lo stato futuro per tutti i valori di op eccettuato **lw**, **sw**, **tipo-R**, **jump** e **beq** sia lo stato 10. Tale condizione è simbolicamente indicata con l'etichetta **altro** per indicare che il campo opcode non corrisponde a nessuno di quelli che compaiono nelle altre etichette degli archi uscenti dallo stato 1.
- **Overflow aritmetico:** il segnale **overflow** viene utilizzato nella macchina a stati finiti modificata per specificare un ulteriore stato futuro a partire dallo stato 7 per il rilevamento dell'overflow.



Una volta trovato l'errore, salvo in memoria lo *STATO* della computazione appena interrotta da un'eccezione, per mantenere i valori corretti nei registri in modo che l'OS intervenga in opportune aree di memoria



Le gestioni via software possono essere gestite anche con le eccezioni vettorizzate (*vectorized interrupt*). Un'eccezione è vettorizzata quando non viene identificata dal programma di gestione dell'eccezioni, quindi invece che caricare l'indirizzo dell'unico gestore viene caricato l'indirizzo di un gestore dell'eccezione apposita (quella corrente).



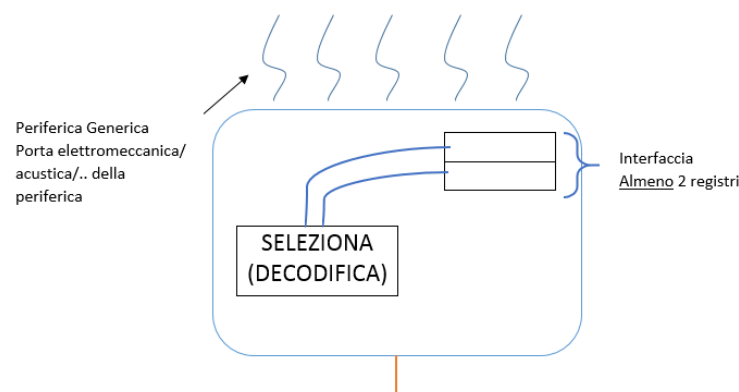
Ma se durante un'eccezione (l'esecuzione del gestore dell'eccezione) avessi un'altra eccezione?

Va prevenuta e gestita, garantendo che non ci siano interruzioni durante la gestione di un'eccezione disabilitando lo status dell'*interrupt*. In poche parole, si dà priorità al first-in-first-out (prima gestione eseguita viene risolta).

Gestioni I/O



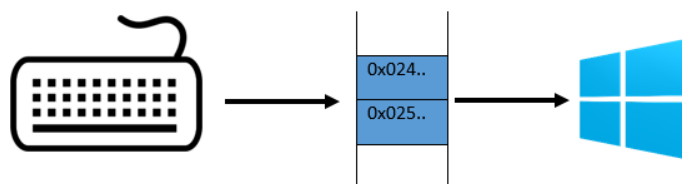
Memory Mapped I/O



I due registri di interfaccia sono **reg.Data** e **reg.Stato** che servono per il passaggio di dati tra **periferiche** e **bus dati** esso può essere letto o scritto (input o output).

La periferica possiede un circuito di selezione che risponde ad un determinato indirizzo come accesso in memoria.

Per esempio la tastiera ha degli indirizzi specifici riservati, che l'OS utilizza per fare la **load word** per permettere il corretto dialogo tra periferiche hardware esterne con quelle interne.



Spazio Indirizzamento I/O

Oltre alla `memory_mapped` ci sono anche alcune macchine che posseggono un bus separato per le periferiche, con cui utilizza delle istruzioni specifiche per il dialogo.

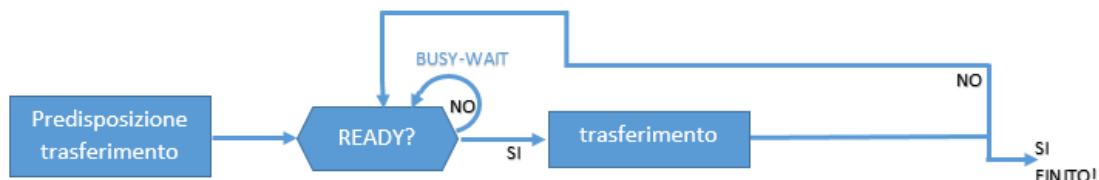
Per esempio, se devo leggere un dato dall'`hard_disk` non potrò utilizzare una `load word`, ma altre istruzioni apposite.

Le periferiche possiedono diverse velocità, questo può causare problemi perché si rischierebbe di sovrascrivere i dati, perciò si hanno due unità di misura:

- **Banda Passante** : (dati/t) cioè quanti dati passano al secondo.
- **Criticità** : il tempo e la possibilità di sovrascrittura dei dati.
 - **Latenza** : è verificata se non si è abbastanza rapidi da prelevare il dato prima che sia sovrascritto.
LATENZA CRITICA: elevato tempo e possibilità di sovrascrittura LATENZA BASSA: i tempi di sovrascrittura sono meno drastici della *latenza critica*, ma comunque pericolosi.
LATENZA INFINITA : alcune periferiche non sovrascrivono il dato finché il dato non viene prelevato.

Tecniche di controllo del programma

Queste tecniche sono utilizzate per evitare la *latenza*. Si usano i due registri di interfaccia per verificare che la periferica sia pronta, in caso contrario, si resta in attesa.



Predisposizione trasferimento : `var dove, var quanti`

Ready : sposto "stato" in un registro generale, `lw $stato, $x`, controllo con un `beq $x, "BORDELLO"`

Trasferimento : `lw e sw, "dove"++ , "quanti"--, quanti==0?`

Poniamo l'esempio di cercare un file di nome "**bordello**" nell'`hard_disk`.

creo due variabili di nome "**dove**" che indicherà la locazione nell'`hard_disk` e "**quanti**" il numero di file da prelevare.

Ad ogni iterazione del "**ready?**" si effettua un controllo tramite una **branch** tra il file prelevato e il file che cerco, se coincidono viene trasferito e le variabili vengono incrementate le locazioni e decrementate le

quantità, il file viene infine trasferito. Se il numero di file da trasferire equivale a 0, il trasferimento è concluso, altrimenti si ritorna alla fase di "ready?".

La *Latenza* è il tempo che intercorre da quando la periferica non è pronta a quando è pronta. (bit di stato/bit ready 1)

→ PRONTA → DATO → NON PRONTA → FINISCE STAMPA "BORDELLO" → PRONTA
(deve leggere/scrivere)

Durante *busy-wait* è importante che intanto il computer esegua altre istruzioni, esso è un costo in **termini di alimentazione**.

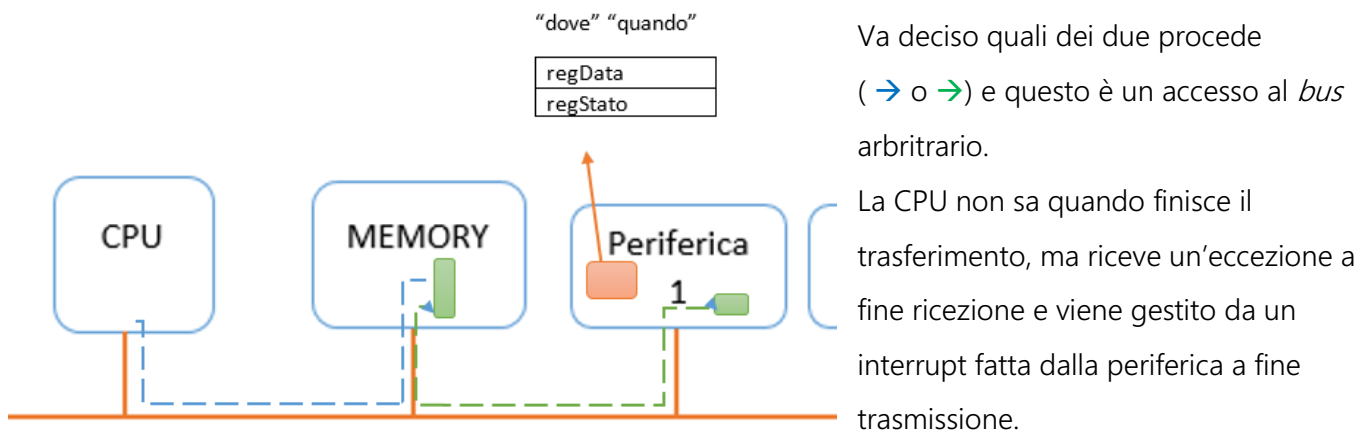
Interruzione programma

Se non è pronto si comporta come l'eccezione e viene gestita dal gestore di eccezioni (vedi s. 35)

La differenza è che entrano in gioco, trattando di periferiche, i tempi di *latenza* e la *banda passante*.

La **banda passante** e la **latenza** è minore (a parità di CPU e clock) questo consente di evitare il *busy-wait* è peggiore in termini di velocità ma ottima per il *multitasking*.

DMA (Direct Memory Access)



*Ti ricordo, a te che stai leggendo, che io(Martin), Samu, Tia e Michi, abbiamo capito solo alla fine che tutta questa cosa non è gestita dal processore ma lo decide il programmatore. Questo è stata omessa a lezione (o forse non l'abbiamo sentito...) rendendoci la vita assai, assai, assai, assai difficile. Quindi mentre leggi ricorda che è tutto a discapito del programmatore. Il programmatore gestisce lo stack, il programmatore gestisce il caricamento del valore dei registri in memoria, il programmatore gestisce **every - fuckin' - thing**.*

Quando inizia l'esecuzione delle istruzioni all'interno del processore, vengono salvati i contenuti dei registri $\$sx$ ($\$s0$, $\$s1$, $\$s2$, $\$s3$, ... , $\$s7$) in memoria mentre i contenuti dei registri $\$tx$ ($\$t0$, $\$t1$, $\$t2$, ... , $\$t9$) non vengono salvati perché sono utilizzati dal processore con i registri $\$a0$ - $\$a3$, parallelamente all'utilizzo dei registri riservati $\$v0$ - $\$v2$. A fine procedura vengono ripristinati i valori dei registri $\$sx$ ($\$s0$, $\$s1$, $\$s2$, $\$s3$, ... , $\$s7$).

Procedure Innestate

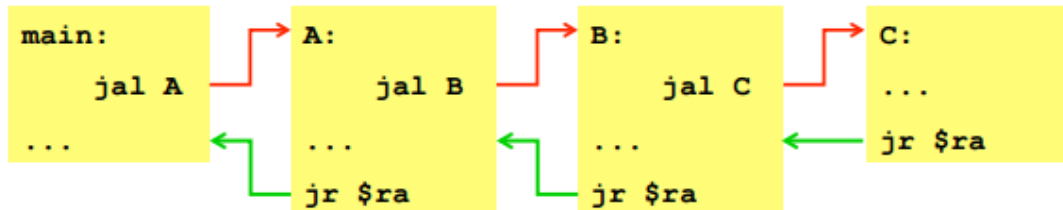
Le procedure innestate, o procedure **annidate**, sono procedure *"non foglia"*. esse richiamano altre procedure che devono salvare nello **stack**:

- L'indirizzo di ritorno (MIPS: registro $\$31$: $\$ra$)
- Gli argomenti di input alla procedura: $\$a0$ - $\$a3$
 - Se la procedura chiamata richiede parametri, i registri $\$a0$ - $\$a3$ potrebbero venire riscritti.
- Il record di attivazione (es. Variabili locali non temporanee [$\$sx$])

```

main:  ...
      li $a0, 3
      jal A
ra_M:  sw $v0, 0($t0)
A:     ...
      addi $sp,$sp,-32
      sw $a0 8($sp)
      ...
      li $a0, 24
      jal B
ra_A:  sw $v0,4($t2)
      add $a0, $s0, $a0
      ...
      addi $sp,$sp,32
      jr $ra
B:     addi $sp,$sp,-24
      ...
      li $a0, 68
      jal C
ra_B:  sw $v0,0($t3)
      ...
      jr $ra
C:     addi $sp,$sp,-12
      ...
      move $a0, $s4
      sw $v0, 4($t2)
      ...
      jr $ra

```



La procedura è gestita esplicitamente:

1. Vengono modificati e poi ripristinati i **valori-argomento** contenuti nei registri **\$a0 - \$a3**

SOLUZIONE: salvo **\$a0-\$a3** nello **stack**

1. **Prima** della chiamata (**jal**), salvo **\$a0-\$a3** nello stack(**PUSH**)
2. Chiamata/e a procedure **jal**
3. **Dopo** tutte le chiamate, recupero **\$a0 - \$a3** dallo stack(**POP**)

2. Ripristinare l'**indirizzo di ritorno** dopo le chiamate (contenuto del registro: **\$ra**)

SOLUZIONE: salvo **\$ra** nello **stack**

1. **Prima** della chiamata(**jal**), salvo **\$ra** nello stack (**PUSH**)
2. Chiamata/e procedura: **jal**
3. **Dopo** tutte le chiamate, recupero **\$ra** dallo stack (**POP**)

Uso dello stack

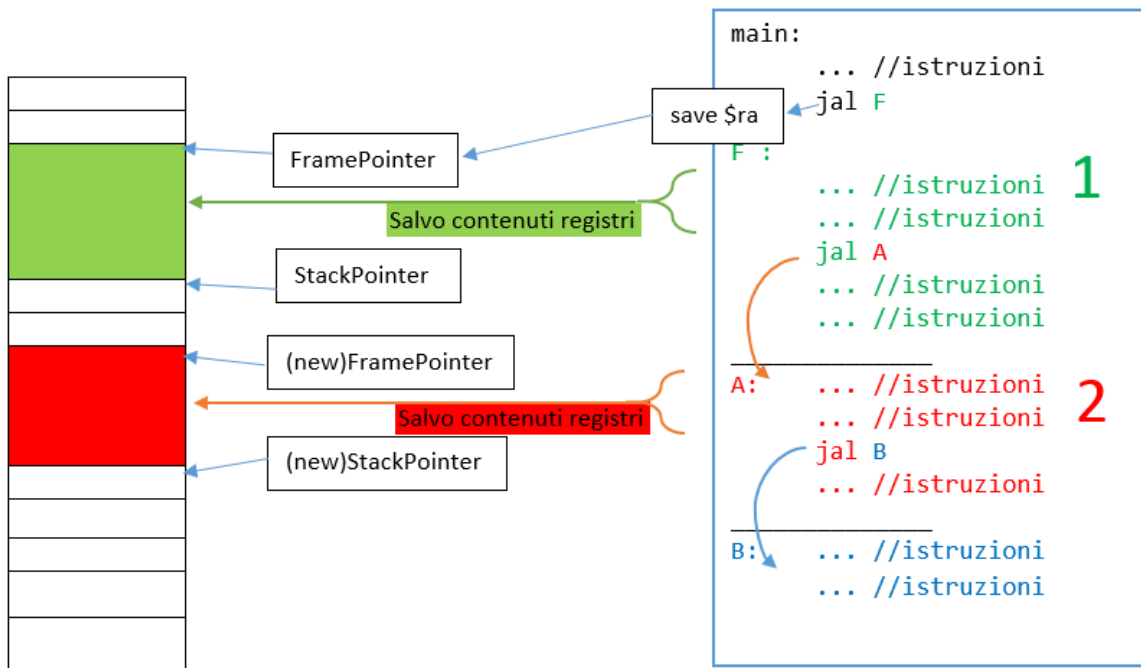
Se devo passare più di 4 parametri (**\$a0 - \$a3**) i restanti vanno in cima allo stack (**PUSH**), nella procedura so quanti parametri servono → **POP** dallo stack.

Il **framePointer** è un registro (**\$fp**) e viene calcolato tramite lo **stackPointer(\$sp)** più la dimensione dello stack meno 4

$$\text{\$fp} = \text{\$sp} + (\text{stackSize} - 4)$$

Lo **stackPointer** punta all'inizio dello spazio allocato dal *chiamante*.

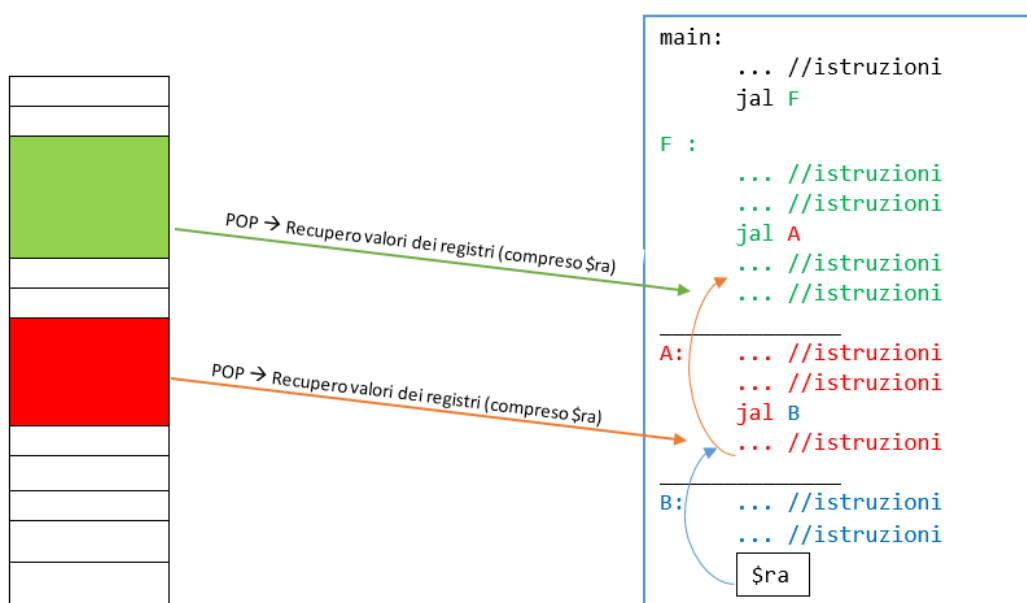
Procedura Andata



1 Quando la procedura chiamante "F" esegue le sue istruzioni e arriva al "jal A" salva i contenuti dei registri allocando lo stack e calcolando il **framePointer** e lo **stackPointer**, dopo esegue il "jal A".

2 Ora che sono nella procedura "A" eseguo ancora le istruzioni (i registri ora sono liberi perché sono salvati nello stack precedentemente allocato dal chiamante F), quando arriva al "jal B" calcola il nuovo **framePointer** e lo **stackPointer** (che è uguale alla dimensione dello stack [in indirizzo]), dopodiché salta liberando di nuovo i registri.

Procedura Ritorno



Quando torno indietro dalla procedura chiamata, recupero i registri **\$fp** e **\$ra** (più i vari valori dei registri salvati precedentemente) per riprendere le istruzioni del chiamante, procedo così finché non torno al main.

Pipeline

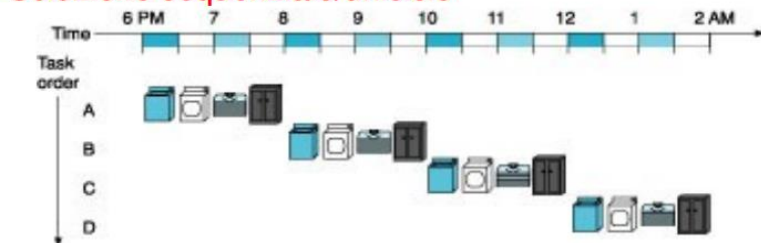
Il **pipeline** è una tecnica di implementazione hardware di un insieme di istruzioni che preve la sovrapposizione temporale dell'esecuzione delle diverse istruzioni.

Tutti gli stadi della **pipeline** sono attivi contemporaneamente, purché si abbiano risorse separate per ciascun stadio.

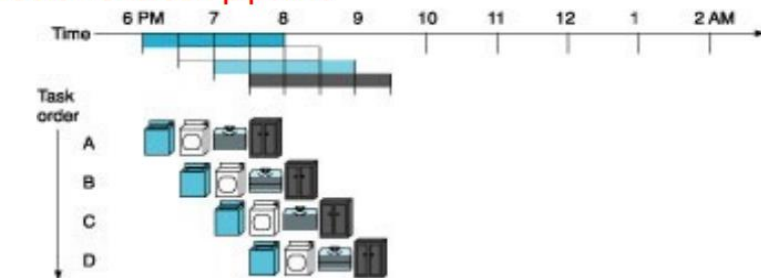
Se consideriamo (come nel libro), l'esempio del bucato, abbiamo che l'approccio basato sul concetto di **pipeline** richiede meno tempo rispetto ad un approccio sequenziale:

1. Quando la lavatrice finisce il primo ciclo di lavaggio e la biancheria viene trasferita all'asciugatrice, si può riempire subito la lavatrice con il secondo carico di biancheria sporca
2. Dopo che il primo carico di biancheria è stato asciugato, si può iniziare l'operazione di stiratura, e si può spostare il carico bagnato nell'asciugatrice e mettere il successivo carico di biancheria sporca in lavatrice (che è già al terzo ciclo).
3. E così via..

Soluzione sequenziale/uniciclo



Soluzione con pipeline



Il paradosso è che il tempo necessario per lavare, asciugare, stirare e mettere nell'armadio un singolo calzino non viene affatto ridotto, quindi non riduce il tempo di esecuzione di ogni istruzione, ma il tempo totale quando ho molti carichi di biancheria da lavare, in quanto la **pipeline** aumenta il *throughput* (cioè il numero delle istruzioni eseguite in un'unità di tempo) della nostra lavanderia. Il motivo per cui le **pipeline** sono più veloci è che tutte le unità lavorano in parallelo, quindi si possono fare più cicli di istruzioni in minor tempo.

Se tutti gli stadi richiedessero circa lo stesso tempo e ci fosse abbastanza lavoro da fare, l'incremento di velocità dovuto alla pipeline sarebbe pari al numero dei suoi stadi (nel caso della lavanderia, 4).

Quindi, se devo fare 20 bucati in sequenza ci metterei 20 volte il tempo di un singolo bucato, mentre in pipeline solo 5 volte il tempo di un singolo bucato, riducendo drasticamente i tempi.

Gli stessi principi si applicano ai processori, dove ciò che si vuole porre in pipeline è l'esecuzione delle istruzioni (**MIPS**) che richiedono, tipicamente, 5 passi:

1. *fetch* : caricamento dell'istruzione dalla memoria.
2. *decode*: lettura dei registri e codifica delle istruzioni (simultanee)
3. *execute*: esecuzione delle istruzioni.
4. accesso ad un operando nella memoria dati
5. scrittura di un risultato nel registro.

Confronto fra le prestazioni di un processore a singolo ciclo e quelle di un processore con pipeline

Per rendere più chiaro il discorso consideriamo una pipeline ben precisa, limitando l'attenzione a otto istruzioni : **load word**, **store word**, **add**, **sub**, **and**, **or** , **set less then** e **branch equal**.

Tipo istruzione	Lettura dell'istruzione	Lettura dei Registri	Operazione con la ALU	Accessi ai dati in memoria	Scrittura nel register_file	Tempo totale
load word(lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
store word(sw)	200 ps	100 ps	200 ps	200 ps		700 ps
Formato R(add,sub,and,or,slt)	200 ps	100 ps	200 ps		100 ps	600 ps
Salto Condizionato	200 ps	100 ps	200 ps			500 ps

Si prenda in considerazione che i multiplexer , l'unità di controllo, l'accesso al PC e l'SGN EXT non introducano ritardo

La prima cosa che va considerata nel singolo ciclo è la durata **dell'istruzione** più lenta (che è la lw a 800 ps), anche se alcune istruzioni richiedono solo 500 ps, nel caso della pipeline si deve considerare la durata **dello stadio** più lento (200 ps), anche se alcuni stadi richiedono 100 ps. La pipeline offre comunque un miglioramento delle prestazioni di quattro volte: l'intervallo di tempo tra la prima e la quarta istruzione è infatti 3 x 200 ps, ossia 600 ps. Per capire come si è arrivati a questa deduzione basta guardare la formula che fornisce l'incremento della velocità prodotta dalla pipeline. Se gli stadi sono perfettamente bilanciati, il tempo di esecuzione delle istruzioni in un calcolatore con pipeline è pari a:

$$\text{Tempo tra due istruzioni con pipeline} = \frac{\text{Tempo tra due istruzioni senza pipeline}}{\text{Numero di stadi della pipeline}}$$

In condizioni ideali l'incremento di velocità dovuto alla pipeline è uguale al numero dei suoi stadi: una pipeline a cinque stadi sarebbe quindi quasi cinque volte più veloce.

In poche parole, la formula sopra ti dice che una pipeline a cinque stadi dovrebbe diminuire di quasi cinque volte il tempo di esecuzione della versione senza pipeline, (nel nostro esempio era 800 ps), cioè basterebbe un clock di 160 ps ($800/5 = 160$). Purtroppo però, non è sempre così, in questo caso noi abbiamo detto che gli stadi erano perfettamente bilanciati, cioè impiegavano tutti gli stessi tempi se così fosse il pipeline sarebbe più efficace di quanto non lo è. Per questo che anche il tempo di fare un ciclo di bucato in pipeline è lo stesso che fare un calzino, perché anche se stirare il calzino è lo stadio che ci piega di meno, noi dobbiamo per forza aspettare lo stesso tempo che ci mette lo stadio più lungo. Quindi, essendo gli stadi non sempre perfettamente bilanciati si introduce nel pipeline dei tempi che si aggiungono ai tempi di esecuzione, e il tempo di esecuzione per la singola istruzione in un calcolatore con pipeline sarà maggiore del minimo teorico, quindi l'incremento di velocità sarà minore del numero di stadi.

Per porre un esempio in modo che sia totalmente comprensibile ciò che si è scritto usiamo un grafico (*nota: usaremos come esempio una load word*).

	200		400		600		800		1000		1200		1400		1600		Tempo
Ordine di esecuzione del programma (sequenza delle istruzioni)																	
lw \$1, 100(\$0)	Fetch Istruzione	Reg	ALU	Accesso dati	Reg												
	800 ps																
lw \$2, 200(\$0)						Fetch Istruzione	Reg	ALU	Accesso dati	Reg							
...											800 ps						

Notare che quando si parla di ciclo singolo, ogni ciclo di istruzione impiega la somma dei tempi di ogni stadio, quindi 800 ps come abbiamo detto prima.

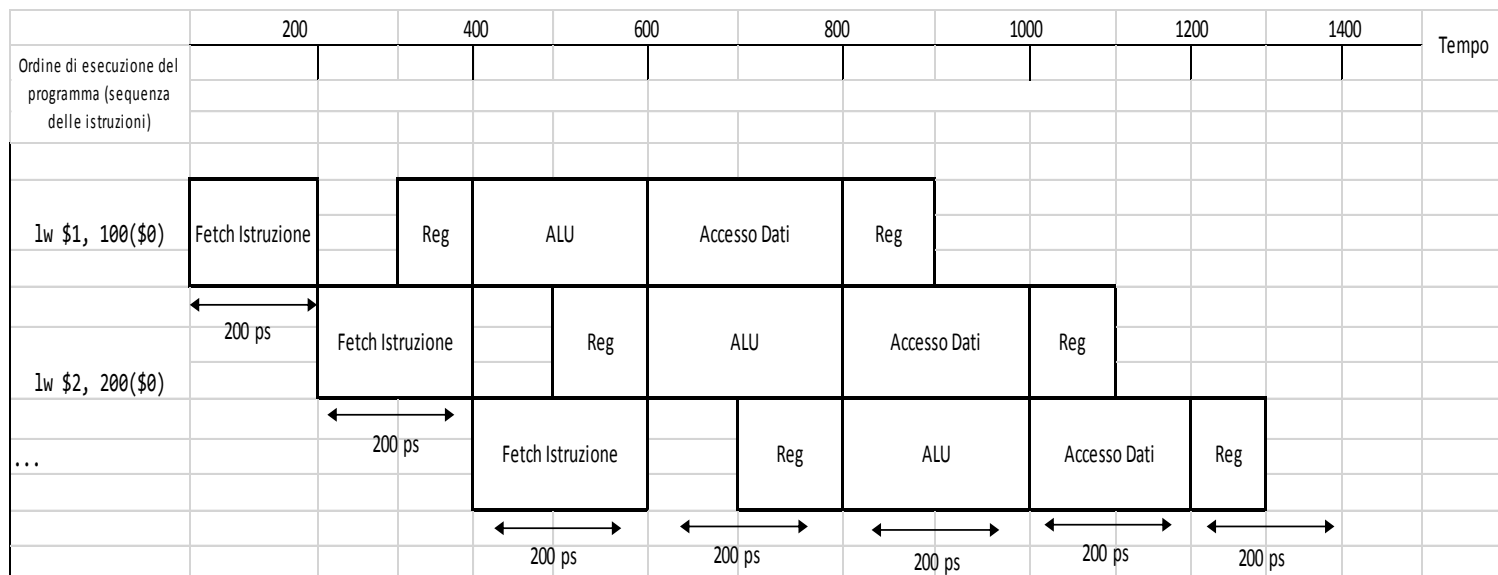
In *pipeline* invece, si adegua tutto al tempo dello stadio più lento (in questo caso *fetch – ALU* e *Accesso ai Dati*) e ne soffre per esempio l'accesso ai registri, che deve stare in *wait* finché non riprende la *fetch*. Si nota però un notevole miglioramento:

Nel *single-cycle* il tempo di 3 cicli di istruzione era 800×3 , quindi 2400 ps, in *pipeline* invece il ciclo singolo soffre perché ci mette ben 200 ps in più rispetto al *single-cycle* questo è dovuto al bilanciamento dei tempi di ogni stadio. Ma già a 3 cicli guadagna 1000 ps, 2400 ps del *single-cycle* contro i 1400 del *pipeline*!.

Ora torna utile la formula:

$$\frac{2400 \text{ ps}}{1400 \text{ ps}} \approx \frac{800 \text{ ps}}{200 \text{ ps}} \cong 4$$

Quindi abbiamo una diminuzione del tempo medio di un fattore 4.



Ricapitolando, la tecnica del *pipeline* migliora le prestazioni *aumentando il throughput delle istruzioni*, ma *non riduce il tempo di esecuzione della singola istruzione (anzi.)*. Tuttavia è il throughput che interessa visto che un processore non svolgerà mai solo un'istruzione, ma ne esegue a miliardi.

Progettazione dell'insieme di istruzioni per architetture dotate di pipeline

Il MIPS è stato progettato espressamente per l'esecuzione in pipeline, di fatto:

- Tutte le istruzioni MIPS hanno la *stessa lunghezza*, tale proprietà semplifica notevolmente la fase di fetch delle istruzioni nel primo stadio. In un insieme di istruzioni come quelle dell'x86, in cui la lunghezza delle istruzioni varia da 1 byte a 15 byte, l'implementazione del *pipeline* sarebbe molto più complesso anche se le implementazioni più recenti dell'x86 prevedono la traduzione delle istruzioni in microistruzioni più semplici che somigliano a quelle del MIPS e inviano quindi alla pipeline queste semplici microistruzioni.
- Le istruzioni MIPS adottano un numero ridotto di formati diversi e i formati dei registri sorgente sono tutti nella stessa posizione in qualunque tipi di istruzioni, questo permette al secondo stadio di iniziare a leggere il register file mentre l'unità di controllo sta determinando il tipo di istruzione letta. Se così non fosse bisognerebbe spezzare ulteriormente il secondo stadio portandoli ad un totale di sei.
- Nel MIPS gli operandi residenti in memoria possono comparire nelle istruzioni di load e di store, questo vincolo permette di utilizzare lo stadio di esecuzione per calcolare l'indirizzo in memoria, per accedere poi alla memoria nello stadio successivo, perché se si potessero effettuare delle operazioni sugli operandi residenti in memoria (come nell'x86) gli stadi 3 e 4 dovrebbero essere espansi a 3, uno per il calcolo dell'indirizzo, uno per l'accesso ai dati in memoria e un terzo per l'esecuzione delle operazioni.

- L'allineamento degli operandi in memoria è obbligatorio nel MIPS, questo evita di dover considerare che in ogni singola istruzione di trasferimento dati abbia bisogno di due accessi alla memoria, quindi con un colpo di clock si possono trasferire dati tra processore e memoria, e quindi in un singolo stadio della pipeline.

Gli Hazard nelle pipeline

In una pipeline si possono verificare situazioni in cui l'istruzione successiva non può essere eseguita nel ciclo di clock immediatamente successivo. Tali eventi sono detti **hazard** (o **criticità**) e possono essere di tre tipi

Hazard strutturali

Il primo tipo di hazard è l'**hazard strutturale**, esso si verifica quando le risorse hardware presenti non sono in grado di supportare la combinazione di istruzioni che si vorrebbe eseguire nello stesso ciclo di clock.

Fortunatamente il MIPS è stato progettato appositamente per essere eseguito in pipeline. Si supponga però di disporre di una singola memoria anziché di due. Se nella pipeline ci fosse una quarta istruzione (*Accesso ai Dati*) la prima istruzione che deve eseguire il *fetch* dovrebbe prelevare dati mentre vengono salvati nel medesimo ciclo di clock! Questo è un caso di **hazard strutturale**.

Hazard sui dati

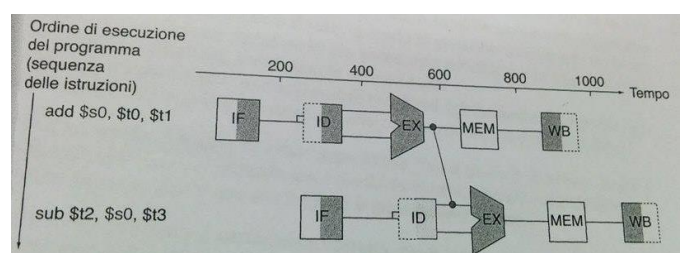
Un **hazard sui dati** si verifica quando la pipeline deve essere messa in stallo perché uno stadio deve attendere che termini l'elaborazione in un altro stadio della pipeline. In un calcolatore dotato di pipeline, gli hazard sui dati nascono quando un'istruzione dipende dal risultato di un'istruzione precedente che si trova ancora all'interno della pipeline. Supponiamo, per esempio, di avere un'istruzione di somma seguita da una sottrazione che utilizza il risultato della somma:

```
add $s0, $t0, $t1
```

```
sub $t2, $s0, $t3
```

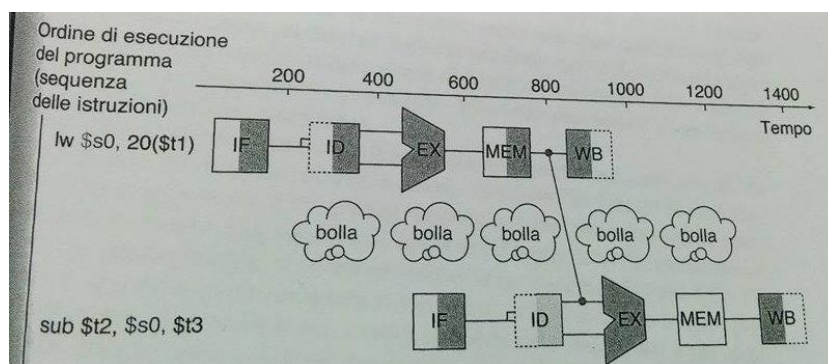
Se non si prendessero delle contromisure, gli **hazard sui dati** potrebbero causare tanti stalli della pipeline: l'istruzione *add* non scrive il proprio risultato prima del quinto stadio, e ciò significa che si dovrebbero perdere tre cicli di clock. Queste dipendenze sono troppo frequenti e il ritardo che viene introdotto è troppo lungo per poter sperare che il compilatore lo risolva. La soluzione più quotata è basata sulla tecnica del **bypassing** o **propagazione** e consiste nell'aggiunta di un

circuito che fornisce in un punto dell'esecuzione il dato mancante prendendolo da un risorsa interna, cioè dall'ALU. Infatti, appena il risultato della somma viene generata dall'ALU questo viene utilizzato come ingresso all'ALU per l'operazione di sottrazione.



Propagazione nel caso di due o più istruzioni

Nell'immagine sopra si nota che la propagazione del risultato è gestita ottimamente, purtroppo però non è ubi gradi di evitare gli stalli di una pipeline. Supponiamo di avere come prima istruzione la lettura dalla memoria a \$s0 (load), anziché una add. Il dato sarebbe disponibile *dopo* il quarto stadio della prima istruzione, ossia troppo tardi per essere utilizzato come *input* del terzo stadio della sottrazione. Di conseguenza, anche disponendo di un meccanismo per la propagazione, dovremmo comunque imporre uno stallo della durata di un ciclo di clock a causa di un **hazard sui dati di una load** (*load-use data hazard*). L'immagine sotto mostra la situazione citata, chiamato formalmente **stallo della pipeline** ma spesso indicata con i termine **bolla** (*bubble*). Il metodo più indicato per evitare uno stallo della pipeline è la **riorganizzazione del codice**, quindi è ancora a discapito del programmatore la gestione del flusso della pipeline.



Hazard sul controllo

Il terzo tipo di hazard è rappresentato dagli **hazard sul controllo** o **hazardi sui salti condizionati**. Questo hazard avvengono quando bisogna prendere una decisione in funzione del risultato dell'esecuzione di un'istruzione e altre istruzioni sono già state avviate all'esecuzione.

In un calcolatore l'equivalente di un'operazione di scelta è l'istruzione di salto condizionato. Nota che la lettura dalla memoria istruzioni dell'istruzione che segue la branch viene fatta nel ciclo di clock immediatamente successivo al prelevamento della branch; tuttavia la pipeline non ha alcun modo di sapere quale sarà l'istruzione successiva perché *ha appena ricevuto* l'istruzione di branch dalla memoria, una soluzione sarebbe mettere in stallo la pipeline immediatamente dopo aver caricato dalla memoria un'istruzione di branch e attendere fino a che la pipeline non abbia determinato il risultato del confronto prima di calcolare l'indirizzo dell'istruzione successiva.

La Cache

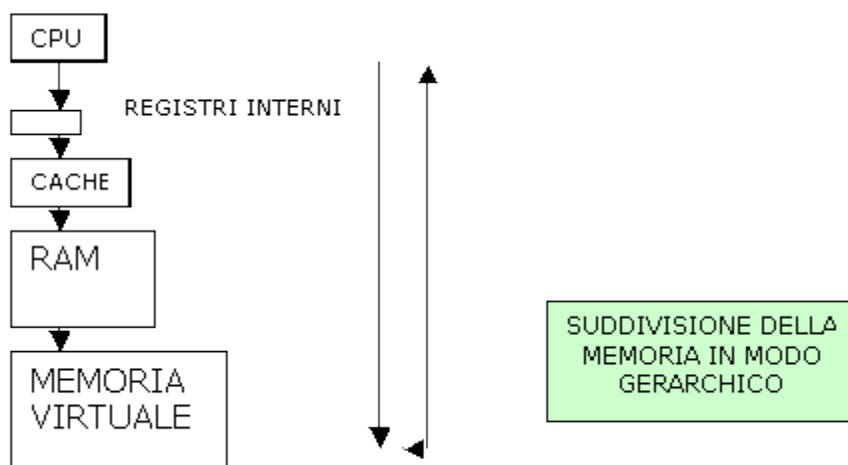
La memoria di un calcolatore è formata da varie sequenze di celle di dimensioni variabili e può essere suddivisa in diversi tipi: cache, memoria principale e memoria di massa.

La memoria interna è composta in piccola parte da memoria di sola lettura (**ROM**) e da memoria di lettura-scrittura (**RAM**) che come visto in precedenza può essere di vario tipo: DRAM, SRAM, SSRAM, etc..

La memoria di massa invece è un tipo di memoria non volatile che permane una volta scritta e può essere ad esempio un *hard-disk*, un *cd-rom*, etc..

La RAM, sebbene sia veloce lo è abbastanza da riuscire a seguire i moderni processori, quindi il processore perderebbe troppo tempo ad aspettare l'arrivo dei dati dalla memoria. Al fine di ottenere migliori prestazioni con il minor costo possibile si è venuta a creare la necessità di suddividere la memoria in modo gerarchico con sopra le memorie più piccole e veloci (**cache**), e sotto con memorie più ampie ma più lente (RAM, **memorie virtuali**).

Tipicamente la memoria **cache** è dai 20 alle 100 volte più piccola della memoria centrale ma è anche 5 o addirittura 20 volte più veloce.



L'utilità della cache è quindi quella di far rimanere in memoria i dati che il processore usa più spesso, e la prima volta che la CPU carica dei dati dalla memoria centrale, questi vengono caricati anche sulla cache e le volte successive, i dati possono essere letti dalla cache più velocemente invece che dalla memoria centrale che è più lenta.

Le cache sono suddivise in due livelli, spesso si fa riferimento alla cache di *secondo livello (L2)*, chiamata anche cache esterna, mentre la cache di *primo livello (L1)* sono quelle integrate direttamente sulle CPU. La cache L2 è da sempre composta da SRAM. Il tempo di accesso medio va dai 10 ai 25 nanosecondi (*ns*). La maggiore velocità è dovuto al fatto che non è necessario fare il refresh del contenuto della cella visto che quest'ultima è composta (come ben sai!) da due transistor collegati in modo da avere solo lo stato 0 o 1. La cache funge da

memoria di transito fra la RAM centrale e la CPU. Ogni volta che quest'ultima preleva informazioni dalla RAM, ne è riprodotta una copia che finisce nella cache per essere immediatamente reperibile per qualsiasi accesso successivo, nella cache sono copiate inoltre le informazioni che in memoria sono adiacenti al dato appena richiesto così che nelle successive operazioni la CPU non debba più accedere alla RAM ma possa servirsi direttamente dalla cache, ben più veloce.

Si segue un **principio di località** :

- **SPAZIALE** : se nell'esecuzione di un programma la CPU accedere ad una cella di memoria (contenente istruzioni), è molto probabile che le celle vicine siano le prossime a cui deve accedere. Quindi si carica un "blocco" di istruzioni seguenti (alcune volte può risultare inutile ma nella maggior parte no ☺) così da risparmiare accessi in memoria.
- **TEMPORANEO** : Se un programma ha fatto accesso ad una zona di memoria, è molto probabile che poco dopo riacceda a quella zona.

Coerenza della Cache

Per capire l'efficienza della cache si introducono due misure:

- **HIT** : cioè quante volte la CPU legge dalla cache senza coinvolgere la RAM
- **MISS**: il numero di volte che il dato non è reperibile nella cache e bisogna perscarla dalla RAM.

L'efficienza della cache viene misurata in *HIT_RATE* cioè la percentuale di HIT in confronto con il numero di accessi(MISS).

$t = \text{HIT_RATE}$

h = tasso di HIT

t_c = tempo di accesso alla cache

m = tasso di MISS ($m = 1 - h$)

t_p = tempo di penalizzazione, esso è dato dal tempo di accesso in memoria (t_{accM} .) più il tempo di trasferimento (t_{trasf})

$$t = h * t_c + m * t_p$$

$$t = h * t_c + (1 - h) * (t_{accM} + t_{trasf})$$

this is the end, my friend.

Autori:

Lorenzo Genny Senesi	<i>Per l'idea e lo sbatti</i>
Martin Tahiraj	<i>Perché a parte le inutili e scontate 20 pagine iniziali scritte da Genny il resto l'ha scritto tutto lui, e senza di lui non ci sarebbe l'indice, i disegni, il design e l'espressione artistica che sprigiona questo .doc.</i>
Mirko Rima	<i>Perché un negroni è per sempre</i>
Mattia Pennati	<i>Per il quaderno, gli appunti, i suggerimenti e le spiegazioni.. ma specialmente, per le bestemmie.</i>
Federico Probo(lon)	<i>Per la costante ignoranza</i>
Grein Nuhu	<i>Per la sua presenza scontata</i>
Sara Riva	<i>Per il quaderno e i caffè</i>
Lorenzo Maffioli	<i>Per il suo accento</i>
Michele Perrotta	<i>Per l'opinione e la criticità che serve, hemper.</i>
Pietro Colombo	<i>Per il fatto che ci sopporta la Sara.</i>